

项目简介

OpenHarness 是一个轻量级 Agent Harness（代理基础设施），为 LLM 提供 手、眼、记忆和安全边界。一句话描述：模型是 Agent，代码是 Harness。

核心功能

- Agent Loop — 流式工具调用循环，支持并行工具执行、指数退避重试、Token 计数与成本跟踪
- 43+ 工具 — 文件 I/O、Shell、搜索、Web、MCP 协议全覆盖，每个工具有 Pydantic 输入校验
- Skills 系统 — Markdown 知识按需加载，兼容 anthropics/skills 生态
- Plugin 系统 — 命令 + Hook + Agent 三合一插件架构，兼容 claude-code 插件
- Context & Memory — CLAUDE.md 自动发现注入、MEMORY.md 持久记忆、Auto-Compact 自动压缩
- Governance — 多级权限模式（default/auto/plan）、路径规则、命令拒绝列表、PreToolUse/PostToolUse Hook
- Swarm 协调 — 子代理生成、团队注册、后台任务生命周期、通知系统
- React TUI — 基于 React/Ink 的终端 UI，支持 Markdown 渲染、权限对话框、命令选择器
- 多 Provider — 支持 Anthropic/OpenAI/Copilot/Codex/Moonshot/GLM/MiniMax/Ollama 等
- 项目类型：Agent 基础设施框架
- 开发语言：Python（3.10+）
- 状态：活跃开发中（v0.1.7）

1. Agent vs harness agent

Agent 本质上是一个具有自主决策能力的 AI 程序，它的核心是一个大语言模型，但不仅仅是个模型——它还具备规划能力、记忆系统和工具使用能力。Agent 的关键特征是"动态决策"：LLM 会根据中间结果自己决定下一步做什么，而不是按照预先写好的固定流程走。比如你让一个 Agent"帮我修复这个 bug"，它可能先读代码、再分析问题、再写修复、再跑测试——每一步都是它自己判断后决定的，顺序不是人提前规定好的。

Harness 则完全不同。Harness 是 Agent 的"操作系统"层，它不参与"怎么思考"的决策，只负责"能做什么"的约束和支撑。打个比方，Agent 是大脑，Harness 是身体。大脑决定"我要拿起这个文件"，但身体提供手（工具执行）、提供眼睛（文件读取）、提供记忆（MEMORY.md）、提供安全边界（权限检查）。大脑可以说"我想修改/etc/passwd"，但身体的手会被 Harness 的权限系统拦住——大脑做决策，身体决定能不能执行。

Harness Agent 就是这两层的组合体。在 OpenHarness 里，Agent Loop 引擎（engine/query.py 里的 while 循环）负责智能决策循环——每一轮调用 LLM、解析响应、决定是否执行工具、是否继续下一轮。而 Harness 基础设施层提供 43 个工具的注册和执行、权限治理（敏感路径硬保护、三种权限模式）、记忆管理（四种类型的 memory 文件）、上下文压缩（三级压缩策略）。这两层是彻底分离的：Agent Loop 不依赖特定的 LLM Provider（可以是 Anthropic、OpenAI、任何兼容 API），Harness 层不依赖特定的 Agent 策略（你可以在上面跑 ReAct、跑 Plan-and-Execute、跑任何决策模式）。

举个例子说明这种分离的价值。假设你想把 Agent 从"单 Agent 模式"改成"多 Agent 协作模

式”，在传统框架里你可能要重写整个权限系统、重新注册工具。但在 Harness 架构里，你只需要改 Agent 层的决策逻辑——让它学会“这个任务该派给哪个 Worker”——底层的工具执行、权限检查、记忆管理完全不用动。就像换了个人来开车，但车（Harness）还是同一辆，方向盘刹车都在原地。

面试官可能会追问“为什么不直接用 LangChain 做 Agent？”这个问题很有代表性。LangChain 把推理流程作为核心抽象——Chain 是一串预定义步骤，Graph 是有向图编排多 Agent 流程。OpenHarness 把基础设施作为核心抽象——它不规定 Agent 怎么思考，只规定 Agent 能做什么、有什么资源可用。这两者其实是互补关系：你完全可以用 LangGraph 来编排 Agent 的决策流程（“先分析、再规划、再执行”），但让 OpenHarness 提供底层的工具执行、权限治理、上下文压缩。就像 LangGraph 是“导航软件”（规划路线），OpenHarness 是“汽车操作系统”（提供引擎、刹车、安全限制）——导航说“左转”，汽车执行转向，但如果导航说“撞墙”，汽车的操作系统会拒绝执行

传统 ReAct vs OpenHarness Harness 核心区别

本质差异：传统 ReAct 是“裸奔执行”，Harness 是“安全框架执行”。

维度	传统 ReAct	OpenHarness Harness
执行方式	直接调用工具	7步框架包裹执行
安全控制	无保护	三层权限（敏感路径/模式/规则）
用户干预	无	修改操作需用户确认
输入验证	无	自动 Pydantic 校验
输出处理	原样返回	> 16KB 自动截断保存
状态追踪	只记对话	tool_metadata 记录执行细节
上下文管理	被动等报错	主动三层压缩
会话持久	无	自动保存恢复
扩展机制	硬编码工具	Skills/Plugins/MCP 动态加载
生命周期钩子	无	5种Hook可拦截修改

六大区别：

1. 安全边界：传统无限制，Harness 有敏感路径保护+权限模式+用户确认

2. 执行可控：传统一行执行，Harness 七步流程每步可拦截

3. 状态追踪：传统只记对话，Harness 记执行状态+文件变更+命令历史

4. 上下文管理：传统被动等报错，Harness 主动压缩不中断

5. 用户控制：传统完全自主，Harness 人机协作关键需批准

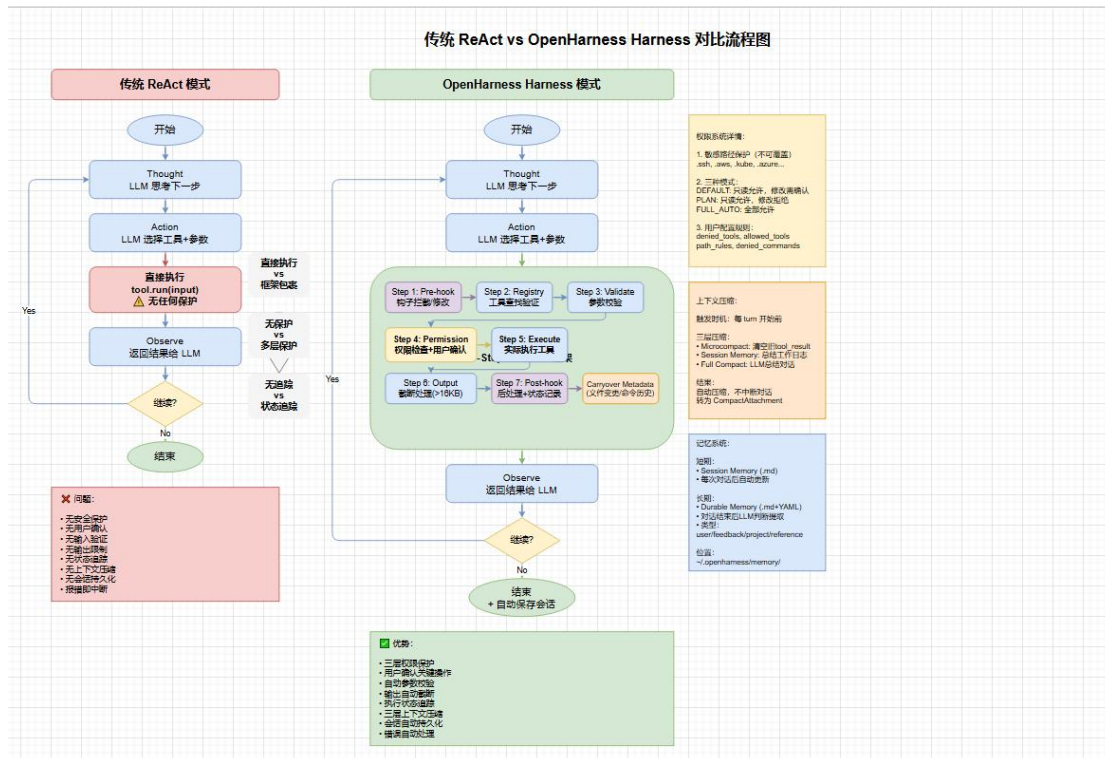
6. 扩展性：传统硬编码，Harness 动态加载+钩子扩展

一句话总结：

Harness 把 ReAct 从“玩具级”提升到“生产级”，给 LLM 装上安全护栏、活动日志、记忆系统。

<以上所有信息均为中兴通讯股份有限公司所有，不得外传>

第 2 页



总体模块解析:

子系统	学习深度	关键知识点
Engine	✓ 深入	Agent Loop 主循环、工具执行 7 步 harness、三层压缩、上下文管理
Tools	✓ 深入	工具查找→输入验证→权限检查→执行→输出截断→Hook 的完整链路
Memory	✓ 深入	四组件 (tool_metadata/messages/Session Memory/Durable Memory)、中文分词搜索、frontmatter 解析
Hooks	✓ 深入	Pre-hook/Post-hook 调度时机、携带 metadata 的 carryover 机制
Permissions	✓ 深入	asyncio.Lock 序列化、permission modal 的单线程设计
Coordinator	✓ 深入	主 agent 与子 agent 的通信、task-notification 消息格式
Tasks	✓ 部分	异步 agent 任务、后台任务管理、输出文件读取

?

可能没深入讨论的内容

子系统	状态	备注
Skills	⚠ 未深入	技能加载机制、技能目录结构、技能的 prompt 组装
Plugins	⚠ 未深入	插件发现机制、MCP 插件集成方式
MCP	⚠ 未深入	MCP server 连接、工具暴露、协议细节

Message:

完整的对话历史（用户说的 + 模型说的 + 工具调用 + 工具结果），每次 API 调用都发送到 LLM，随用户和系统工具响应实时更新，随 Loop 不断增长，太长会被压缩。存放在内存中，对话结束消失。

包含两部分：role: [user]与[assistant] + content: textblock + tooluseblock，工具调用结果写在 user 下

典型的对话历史:

```
messages = [  
    ConversationMessage(role="user", content=[  
        TextBlock(text="帮我做个计划") # 用户文字  
    ]),  
    ConversationMessage(role="assistant", content=[  
        TextBlock(text="好的,我来..."), # AI文字  
        ToolUseBlock(id="tool_1", name="read", input={"path": "xxx.py"}) # 工具请求  
    ]),  
    ConversationMessage(role="user", content=[  
        ToolResultBlock(tool_use_id="tool_1", content="文件内容...") # 工具结果  
    ]),  
    ConversationMessage(role="assistant", content=[  
        TextBlock(text="分析完成..."), # AI回复  
    ]),  
    # ... 继续循环  
]
```

Tool_metadata

工具执行后的状态积累,它不是对话内容。

是一个字典,工具执行过程中累计的状态信息,存在于运行时内存中,随会话结束而消失,不写入文件。**每次工具调用后,都会根据 message 更新。**在 loop 结束后转换为 session memory 短期记忆文件。

tool_metadata是什么

tool_metadata是工具执行过程中积累的状态信息。是一个字典,包含多个字段:

```
tool_metadata = {  
    "read_file_state": {"main.py": {"offset": 0, "limit": 200}},  
    "invoked_skills": ["pytest-config"],  
    "async_agent_tasks": [{"task_id": "abc", "status": "running"}],  
    "recent_work_log": [{"action": "read", "path": "main.py"}],  
    "task_focus_state": {"focus": "重构认证"},  
    ...  
}
```

tool_metadata记录了:

- 读了哪些文件、读了多少行
- 调用了哪些技能
- 创建了哪些子Agent
- 最近做了什么操作
- 当前任务焦点是什么

```
_PERSISTED_TOOL_METADATA_KEYS = (  
    "permission_mode",      # 权限模式  
    "read_file_state",      # 已读取的文件状态  
    "invoked_skills",       # 已调用的技能列表  
    "async_agent_state",    # 异步 Agent 状态  
    "async_agent_tasks",    # 异步 Agent 任务列表  
    "recent_work_log",      # 最近工作日志  
    "recent_verified_work", # 最近验证的工作  
    "task_focus_state",     # 任务聚焦状态  
    "compact_checkpoints",  # 压缩检查点  
    "compact_last",         # 最后一次压缩信息  
)
```

tool_metadata不送给LLM，但会在这些地方使用：

使用场景一：工具执行时

工具执行时，系统把tool_metadata传给ToolExecutionContext：

```
ToolExecutionContext:  
    cwd: 工作目录  
    metadata: tool_metadata (包含read_file_state、task_focus_state等)
```

工具根据metadata决定行为：

- read_file工具知道用户之前读了什么，可以从上次的位置继续
- agent工具知道已有的子Agent，避免重复创建

使用场景二：compact压缩时

压缩生成摘要时，系统用tool_metadata构建Attachment：

- recent_work_log：最近操作，帮助生成摘要
- task_focus_state：任务焦点，摘要中提及
- active_artifacts：外溢文件，摘要中提及

Attachment附加到摘要后面，帮助LLM理解压缩后的上下文。

使用场景三：会话恢复时

会话恢复时，tool_metadata帮助重建工作状态：

- 系统知道读了哪些文件
- 系统知道创建了哪些子Agent
- 系统知道任务焦点是什么

四、为什么看起来重复

Session memory 短期记忆

Session Memory 是一个 Markdown 文件，持久化当前会话的状态，支持 /resume 继续中断的会话。

```
|
|--- session-memory/                                # Session Memory 存储
|   |--- <project-name>-<hash>/                    # 每个项目一个目录
|       |--- default.md                            # 默认会话
|       |--- session-20240115.md                   # 具体会话ID
|       |--- session-20240116.md
```

session memory 是会话快照，保存当前会话的所有关键信息。

session memory 包含: messages 对话历史 + tool_metadata 工具状态 + 其他元数据: 会话 ID、工作目录、模型等。

loop 结束时从 message 生成摘要 + 从 tool_matadata 提取重要内容进行保存

```
{
  "session_id": "sess_abc123",
  "created_at": "2026-06-03T10:00:00Z",
  "cwd": "/Users/project",
  "model": "claude-sonnet-4-6",
  "messages": [
    {"role": "user", "content": "..."},
    {"role": "assistant", "content": "..."},
    ...
  ],
  "tool_metadata": {
    "read_file_state": {...},
    "invoked_skills": [...],
    "async_agent_tasks": [...],
    "recent_work_log": [...],
    "task_focus_state": {...},
    ...
  },
  "summary": null
}
```

可以看到，messages和tool_metadata都在文件里。

长期记忆由 LLM 判断每次对话后是否更新，保留项目、用户偏好

送入 LLM 上下文的内容: **system_prompt + messages:**

Tool_metadata 送入上下文是保证压缩后的 messages 不会丢失关键内容。Tool_meta 转换为 attachment 后塞入 Messages 中

三、Attachment 是什么？

看 `compact/init.py:88-95`:

```
@dataclass
class CompactAttachment:
    kind: str          # 类型: task_focus, recent_work_log, etc.
    title: str         # 标题
    body: str          # 正文内容 (会被渲染成消息)
    metadata: dict     # 元数据
```

`system_prompt`是发给LLM的系统指令，包含：

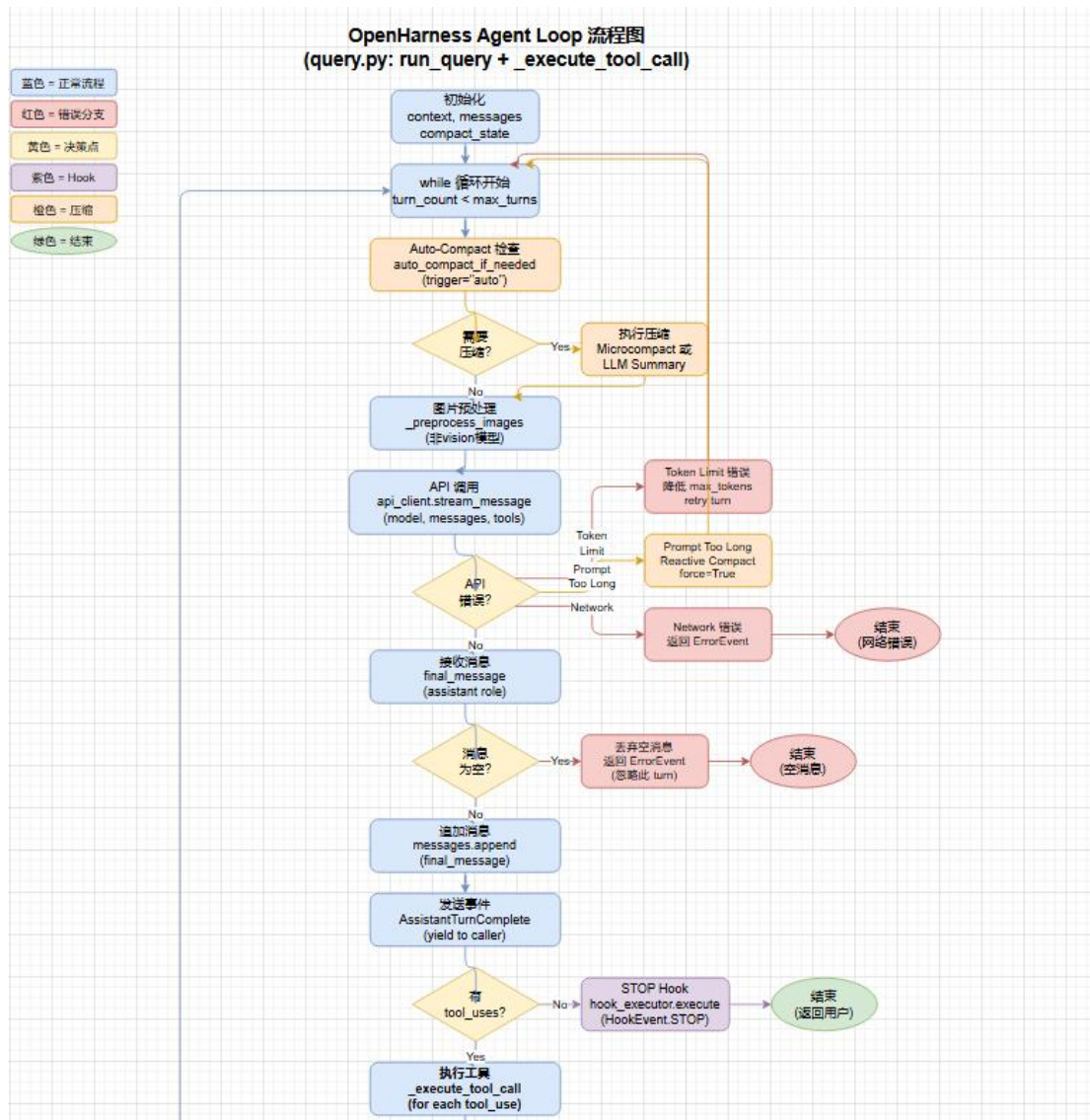
1. Agent的角色定义 (你是一个助手...)
2. 可用工具列表 (read_file、write_file等)
3. 使用规则 (权限说明、操作限制)
4. 已加载的长期记忆 (MEMORY.md里的内容)
5. 当前工作上下文 (任务焦点、工作目录)

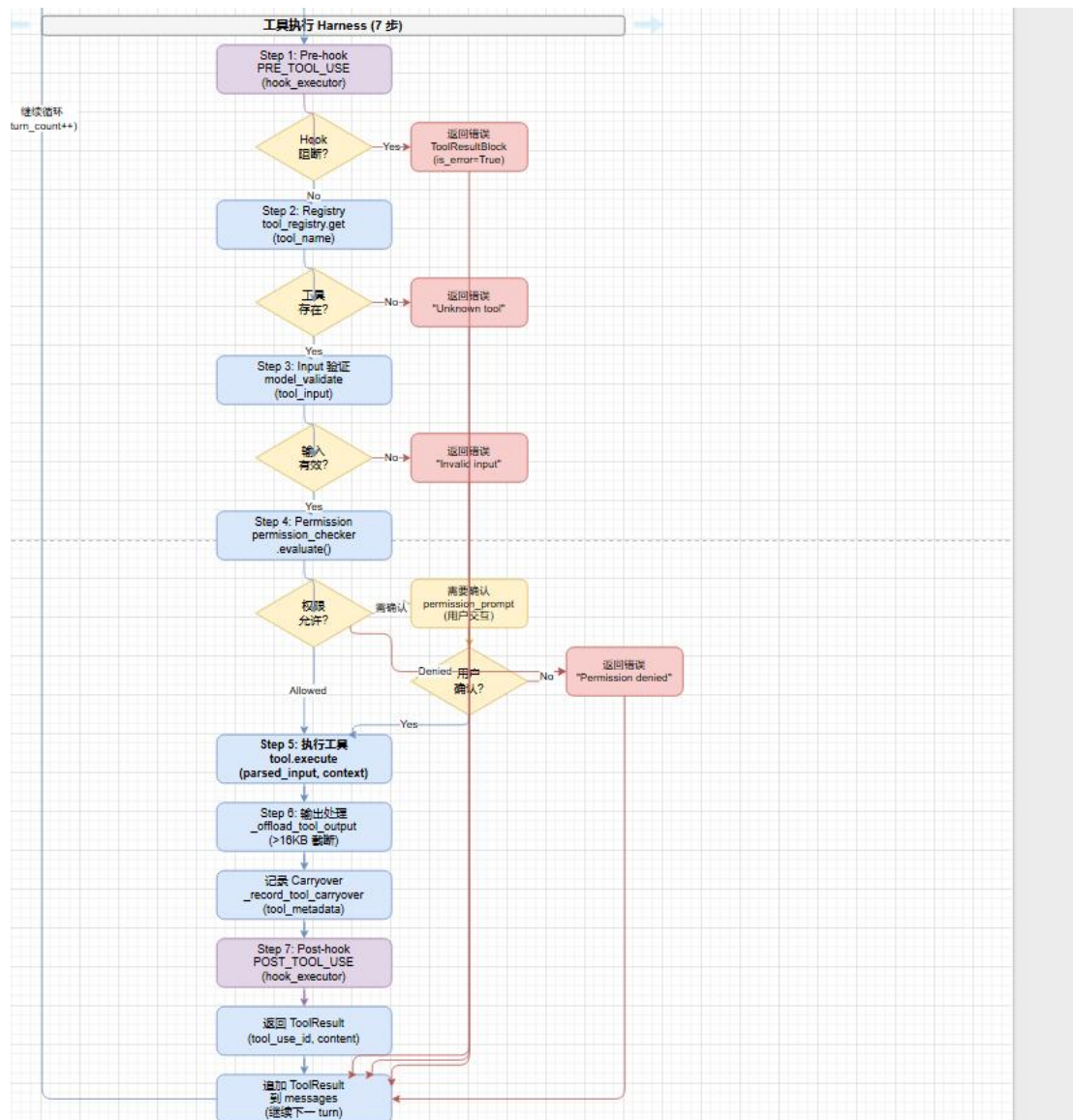
注意：长期记忆 (MEMORY.md) 会注入到`system_prompt`。tool_metadata的一部分信息也可能注入到`system_prompt`。

1. Agent Loop 循环机制

4.agent loop 引擎

Agent Loop 是整个系统的核心引擎，它是一个动态决策循环。与传统程序按预设流程执行不同，Agent Loop 每一步都由 LLM 根据前一步结果自己决定下一步做什么。OpenHarness 的实现位于 `engine/query.py` 的 `run_query` 函数，核心是一个 `while` 循环，`max_turns` 上限 200 轮。





Agent Loop 主循环流程

循环条件: while turn_count < max_turns (默认 max_turns = 200)

Step 1: 循环启动

用户输入一条消息。QueryEngine 把它封装成 ConversationMessage，追加到对话历史 messages。系统同时提取用户意图存入 tool_metadata 的 task_focus_state，并触发 USER_PROMPT_SUBMIT 钩子（用于日志或通知）

Step 2: 上下文治理入口检查 6.上下文压缩

函数: auto_compact_if_needed()

操作: 检查当前消息历史的 Token 估算, 如果超过阈值 (context_window200K - 20K - 13K), 触发 3 级压缩。20K 为当前预估输出内容长度, 13K 为缓存大小。

边界处理: 三层压缩, 压缩的是 message

1. 进行 Microcompact 压缩, 清除旧的 8 种工具 (文件内容、列表等, 结果通常很长, 且信息价值随时间降低) 结果内容为占位符, 保留最近 5 轮。检查上下文长度, 是否仍需压缩, 若不需要可跳过后续步骤。

2. Context Collapse（强制截断），截断单个超长文本块，防止单个 Block 占用过多 tokens。将上下文划分为 older 与 newer(最近 6 轮)，对 older 中的超长文本块（>2400）进行截断（一轮对话包含多个文本块，文字内容、工具调用、工具结果、图片……），具体为保留后 500+前 900token，其他为一个占位符。若不需要 → 返回 messages，跳过后续步骤。
3. 进行 Session Memory 压缩，用规则生成结构化摘要（不调用 LLM，只对除了最近 12 轮外的上下文进行，遍历 older 中的每条消息，提取关键文本，比如用户意图、重要决策、关键发现、工具执行摘要，每条最多 4000token，拼接），成功标准为上下文消息数超过 16、摘要内容有效、压缩后 tokens 或消息数有所减少。成功后返回压缩后的 messages。上下文长度是否小于阈值不是判断成功的标准。若失败继续下一步
4. 进行 Full LLM Compact，将最近 6 轮以外的调用 llm api 生成智能摘要。若压缩后符合上下文长度，返回压缩后的 messages；若不符合，失败次数+1，即 consecutive_failures += 1。
5. 全部失败时，用原始 messages 重试上述 4 步压缩过程，最多三次；若 3 次都失败 → 直接将原始上下文送入 llm，此时会出现 API 错误。
5. API 错误产生时，Reactive Compact 会强制触发（force=True），重新进行上述流程，只能一次，（本次中因为之前失败轮次调用了工具，可能会导致某些块符合压缩条件，且可以将完整压缩后的内容走一次 LLM 试一下能否正常执行，而不是像之前只走压缩前的上下文）。如果 Reactive Compact 也失败，上下文过长 → 告知用户"API error"，退出循环，用户需要手动处理（使用/compact 命令、重开对话、分段完成任务）
若压缩成功 → 更新 messages 列表，consecutive_failures = 0

Q：为什么还需要强制执行一次全部过程，貌似不会减少上下文长度？

A：工具执行产生新块，

Q：压缩后丢上下文吗

A：压缩后上下文保留机制：OpenHarness 通过 Attachment 系统在压缩时从 tool_metadata（Agent Loop 运行过程中各工具/系统实时维护的状态字典，工具执行后就会更新 tool_metadata）提取并携带 8 种关键状态到新上下文——task_focus（当前工作目标 and 下一步）、recent_files（最近读取的文件路径+片段预览）、recent_verified_work（已验证的结论避免重复）、invoked_skills（激活的技能如/review）、async_agents（正在运行的 Worker 任务）、work_log（最近操作记录）、plan（Plan 模式约束）、recent_attachments（用户上传的附件路径）。这些 Attachment 以 user 角色消息形式注入到压缩后的消息列表开头，让 LLM"记得"关键状态，同时 metadata 字段供程序恢复结构化数据。效果：Token 减少 60-80%，但任务连续性不受影响——AI 不会忘记自己在做什么、读过什么文件、哪些已验证、哪些 Worker 还在跑。

Step 2: 图片预处理（多模态适配）

检查模型是否支持多模态

如果支持，跳过预处理（模型能直接看图）

如果不支持，扫描 messages 找出所有 ImageBlock

对每个 ImageBlock 调用 image_to_text 工具

image_to_text 工具用另一个模型（支持多模态的）看图，生成文字描述

把 ImageBlock 替换成文字描述

返回处理后的 messages

错误处理：模型支持多模态 → 跳过预处理；无 vision_model 配置 → 跳过预处理；

image_to_text 工具不存在 → 返回占位文字；转换失败 → 返回错误描述占位

Step 3: API 调用（流式响应）

操作：将 system prompt（包括基础角色定义+长期记忆+由 tool_metadata 格式化后的当前工作状态等）+message 送往 llm

构建请求包含：model, messages, system_prompt, max_tokens, tools

发送 HTTP 请求，返回 SSE 事件流

边界处理：网络错误 → yield ErrorEvent，退出循环等待用户处理；max_tokens 超限 → 自动降级到 provider 限制，重试当前轮；上下文超长（PTL 错误）→ 触发 reactive compact（force=True），压缩后重试；API 重试 → 最多 3 次，指数退避 1s-30s；最终失败 → yield ErrorEvent，退出循环

Step 4: 响应解析与验证

检查 final_message 即 assistant 消息：

message 为空 → 抛 RuntimeError("Model stream finished without a final message")

assistant 消息为空（is_effectively_empty）→ yield ErrorEvent，退出循环

消息处理：

有效时 append 到 messages 历史

yield AssistantTurnComplete 事件

Coordinator 特殊处理：恢复上下文消息

Step 5: 循环终止判断

条件：if not final_message.tool_uses

操作：

LLM 没有请求工具 → 任务完成

执行 STOP Hook

return 退出循环

边界处理：

tool_uses 为空列表 → 视为完成

tool_uses 有内容 → 继续执行工具

Step 6: 工具执行

单工具：串行执行，立即 yield 事件

多工具：并行执行

先 yield 所有 ToolExecutionStarted，执行后 yield 所有 ToolExecutionCompleted

边界处理：

单工具异常 → 捕获，返回错误 ToolResult（is_error=True）

多工具单个失败 → 其他工具正常完成

所有失败 → 返回各自的错误结果

工具超时 → asyncio.TimeoutError，返回超时 ToolResult

Step 7: 工具结果注入

操作：

把工具输出封装成 ToolResultBlock，关联请求 ID。追加到 messages 中作为新的 user 消息。

同时更新 tool_metadata

边界处理：

确保每个 tool_use 都有对应的 tool_result

Anthropic API 要求 tool_use 和 tool_result 必须配对

缺失配对会导致下一个 API 请求被拒绝

Step 8: 下一轮循环

操作：

turn_count += 1

回到 Step 1

循环退出条件

自然完成：tool_uses 为空 → 执行 STOP Hook → return

API 错误：yield ErrorEvent → return

空 assistant 消息：yield ErrorEvent → return

max_turns 耗尽：抛 MaxTurnsExceeded 异常

终止条件（优先级从高到低）：

1. LLM 返回 stop_reason="end_turn"（无 tool_use）

→ 自然终止，任务被认为完成

2. turn_count >= max_turns（默认 200）

→ 抛出 MaxTurnsExceeded 异常

3. API 返回不可重试的错误

→ ErrorEvent + return

4. 空 assistant 消息

→ 丢弃 + ErrorEvent + return（保护会话健康）

5. Auto-Compact 连续失败 3 次

→ 跳过后续压缩，但 Agent 继续运行

防止异常的机制

空消息保护 if final_message.is_effectively_empty():

yield ErrorEvent(message="Model returned an empty assistant message..."), usage

return # 不追加空消息，防止污染历史# 并行工具隔离

raw_results = await asyncio.gather(*tasks, return_exceptions=True)

单工具异常不取消其他工具，确保 API 协议要求（每个 tool_use 有 tool_result）# 消息清洗

sanitize_conversation_messages(messages)

删除孤儿 tool_use/tool_result，防止序列化后的协议错误

系统执行三件持久化操作：

(1) 会话记忆持久化，保存在短期记忆中

把对话历史保存到 ~/.openharness/session_memory/<session_id>.json

下次可以用 /resume 继续会话

(2) 持久记忆提取（可选）

从对话中提取重要信息

写入 MEMORY.md 文件

例如："用户偏好使用 TypeScript"、"项目使用 ESLint 规则"

(3) Auto-dream 后台整理（可选）

触发后台任务，整理长期记忆

在 Agent 长期运行中"积累经验"

空消息保护:

LLM 有时会返回“空消息”:

```
# LLM 返回了一个 assistant 消息，但内容为空
final_message = ConversationMessage(
    role="assistant",
    content=[] # 没有任何内容块
)

# 或者只有空白文本
final_message = ConversationMessage(
    role="assistant",
    content=[TextBlock(text="\n\n")] # 只有空格/换行
)
```

如果不处理，会发生什么？

1. **messages 被污染**: 空消息被迫加到对话历史
2. **下轮 API 调用失败**: 某些 Provider (如 OpenAI) 拒绝接受空消息
3. **对话状态异常**: LLM 可能陷入“空响应循环”
4. **Session 损坏**: 保存后恢复时无法正常工作

丢弃空消息，返回 `errorevent`

2. 工具执行内部流程 (Step 6 展开) 5.工具执行

Step 6.1: Pre-tool-use Hook 8.hook 执行

作用: 执行工具调用前的钩子。禁止特定工具、文件、命令、记录日志等

钩子是一种扩展机制，允许用户在特定事件发生时自动执行自定义代码。在不修改核心代码的情况下，添加自定义逻辑

边界处理:

如果 hook 返回 `blocked=True` → 直接返回错误 `ToolResult`，不执行后续步骤（工具查找、权限检查都不执行）

`blocked` 原因作为 `error content` 返回给 LLM

Step 6.2: 工具注册表查找

作用: 查找工具是否存在。

边界处理:

如果 `tool` 为 `None` → 返回 "Unknown tool: xxx" 错误

表示 LLM 调用了不存在的工具

Step 6.3: Pydantic 输入参数验证

作用: 验证工具输入参数。

执行内容: `parsed_input = tool.input_model.model_validate(tool_input)`

边界处理:

验证失败 → 返回 "Invalid input: xxx" 错误

参数类型错误、缺失必填参数等情况

Step 6.4: 权限检查 7. 权限检查

作用：判断是否允许执行该操作。

执行内容： `permission_checker.evaluate(tool_name, is_read_only, file_path, cmd)`

优先级（从高到低）：

SENSITIVE_PATH_PATTERNS 硬保护 → 直接 deny，不可覆盖

拒绝工具列表 → deny

允许工具列表 → allow

路径规则 glob 匹配 → deny

命令拒绝列表 → deny

Permission Mode 判断

边界处理：

敏感路径 → 直接 deny，无确认

`requires_confirmation=True` → 弹窗让用户确认，用户拒绝 → 返回 "Permission denied" 错误，用户同意 → 继续执行

Plan 模式写入操作 → 直接 deny

Step 6.5: 工具执行

作用：执行工具逻辑。

执行内容： `result = tool.execute(parsed_input, ToolExecutionContext)`

边界处理：

工具内部异常 → 捕获为 `is_error=True` 的 `ToolResult`

执行超时 → 由工具自己处理（不在此层处理）

输出过长 → 进入 Step 6.6 截断

Step 6.6: 输出截断与外溢

作用：处理超长输出。

执行内容： `_offload_tool_output_if_needed()`

处理逻辑：

如果输出超过 **MAX_TOOL_OUTPUT_SIZE**，截断为 **inline** 部分（保留前面），超出部分写入 **tool_artifacts** 文件，记录该文件路径到 **tool_metadata**

边界处理：

正常长度 → 直接返回

超长 → 截断 + 写入文件，返回截断后的 **inline**

Step 6.7: 状态记录

作用：记录工具执行状态。

执行内容： `_record_tool_carryover()`

记录内容： 记录到 **tool_metadata** 的 **carryover** 字段，包括：

工具名、输入、输出、文件路径、是否错误

用途：用于会话恢复时恢复状态。

边界处理：每次执行都记录

Step 6.8: Post-tool-use Hook

作用：执行工具调用后的钩子。

执行内容: `hook_executor.execute(HookEvent.POST_TOOL_USE)`

特点:

Post Hook 无法阻塞 (不能阻止工具结果返回)

只做副作用 (日志、通知)

Hook 失败不影响工具结果

一次调用多个工具时:多工具并行执行,先 `yield` 所有 `Started` 事件,然后用 `asyncio.gather` 并行执行所有工具 (`return_exceptions=True` 保证单个失败不影响其他),等待所有完成后 `yield` 所有 `Completed` 事件,将结果 (包括失败) 转换为 `ToolResultBlock` 注入 `messages`。每个工具独立执行 Hook 和权限检查,失败转换为 `is_error=True` 的 `ToolResult`, 保证 Tool Pair 配对完整

3. 记忆系统

9.记忆系统

记忆在启动 `openharness` 时加载一部分

第一种: 短期记忆 session memory

短期记忆由 `messages` 和 `tool_metadata` 获得, 存储在文件中, 包括 `messages` 中的用户问题、系统回复工具调用, 以及 `tool_metadata` 中的关键状态。

每轮 `Loop` 结束后, 自动更新 `Session Memory` 文件

第二种: 长期记忆

"跨会话持久化的知识"。跨会话意味着: 今天的对话结束了, 明天启动新对话, 知识还在。持久化意味着: 知识存在硬盘上, 不是内存里, 不会因为程序关闭而消失。

用一个叫 `MEMORY.md` 的文件当"目录索引", 每个具体的记忆是独立的 `.md` 文件 (用户偏好、项目约定、重要决策……)。

长期记忆分四种类型, 每种有不同的用途和优先级: `user` 类型存用户画像; `feedback` 类型存用户的纠正指导, 优先级最高; `project` 存项目上下文; `reference` 存外部资源指针。优先级 `feedback > user > project > reference`

设计原理:

不在主 `Loop` 中执行 - 避免干扰正常对话流程

独立 LLM 请求 - 专用 `system prompt`, 禁用工具

只保存"不可推导"的事实 - 不保存可从代码/git/文档获取的信息

限制数量 - 每次最多 3 条, 避免膨胀

敏感信息检测 - `team scope` 需额外验证

幂等性 - 对话中已写过则跳过

```
<project>/openharness/memory/
├── MEMORY.md # 入口文件（索引）
├── user_preference.md # 用户偏好
├── project_conventions.md # 项目约定
├── important_decisions.md # 重要决策
└── ...
```

每个记忆文件是一个 `.md` 文件，包含两部分：

```
---
name: 项目架构说明
description: 记录了项目的整体架构设计
type: project
---
```

正文内容：

项目采用前后端分离架构...

前端使用 React，后端使用 Python...

数据库使用 PostgreSQL...

部分	说明
元数据 (frontmatter)	name、description、type 等信息
正文内容 (body)	详细的信息内容

举个例子，假设你说过"我喜欢 Python"，这条记忆会存成：

文件内容：

name: python_preference

description: 用户偏好 Python 语言

type: user

正文内容：

用户明确表示偏好 Python 语言，不喜欢 Java。

在编写代码时优先使用 Python，除非项目技术栈明确要求其他语言。

文件开头的---包围的部分叫 YAML Frontmatter。YAML 是一种配置文件格式（类似 JSON 但更简洁），Frontmatter 就是"文件头部"的意思。这部分声明了三个关键信息：**name** 是这条记忆的名字（类似 ID），**description** 是描述（AI 用它判断什么时候该加载这条记忆），**type** 是类型（告诉 AI 这条记忆属于哪个分类）。文件的正文部分（---下面的内容）就是记忆的实际内容，用自然语言写的知识。

这样设计的好处是什么？文件存在硬盘上，你关机重启，文件还在。文件是纯文本，你可以用任何编辑器打开看，可以改，可以用 Git 追踪历史。Git 是一个版本控制工具，能记录文件每次修改的变化——比如你一个月前说"我喜欢 Python"，今天改成"我也开始喜欢 Go 了"，

Git 能看到这个变化历史。

```
EXTRACTION_SYSTEM_PROMPT = """
你维护 OpenHarmony 持久记忆。

只保存：
- 对未来有用的稳定事实
- 不能从当前文件、git 历史、文档推导的信息

不保存：
- 秘密/凭证
- 临时状态
- 可推导信息

如果不值得保存，返回 {"memories": []}
"""
```

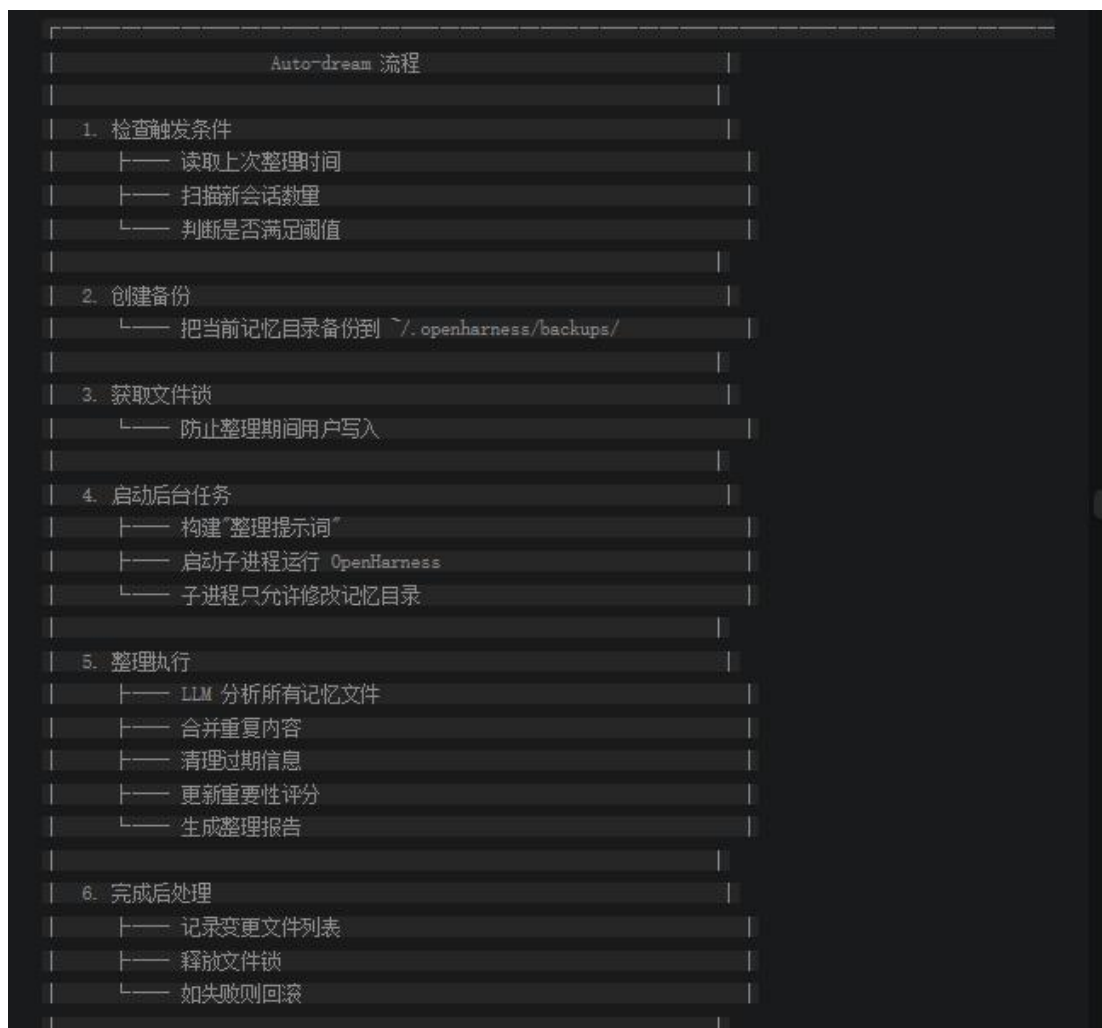
第三种：长期记忆整理 auto-dream

长期运行后，记忆文件可能变得杂乱：重复、过期、碎片化。Auto-dream 在后台自动整理

触发条件

```
# 配置项
auto_dream_min_hours: 24.0 # 距上次整理超过24小时
auto_dream_min_sessions: 5 # 累积超过5个会话
```

只有同时满足这两个条件，才触发整理。



整理提示词示例

你是一个记忆整理助手。请分析以下记忆文件：

记忆目录：`~/.openharness/memory/myproject/`

会话目录：`~/.openharness/session-memory/myproject/`

任务：

1. 合并内容重复的记忆
2. 标记过期的记忆（设置 `disabled=true`）
3. 更新重要性评分
4. 生成整理摘要

约束：

- 只允许修改记忆目录下的文件
- 不允许执行写操作的 `shell` 命令
- 不允许删除文件，只能标记 `disabled`

后台异步执行，不阻塞主对话；备份机制，失败回滚；权限隔离

搜索机制：

当前用的是关键词搜索，不是语义搜索。关键词搜索是你输入一个词，比如"Python"，系统在所有记忆文件里找这个词。如果某个文件里有"Python"这个词，就算匹配。匹配次数越多、位置越重要，分数越高。用两个维度计算分数：元数据和正文，元数据就是 frontmatter 里的 name 和 description，正文就是下面的内容。权重是元数据 2x、正文 1x。

语义搜索不是匹配词，而是匹配意思。比如你搜"数据库"，语义搜索能找到提到"PostgreSQL、MySQL、MongoDB"的记忆，因为这些词的意思都是"数据库"。即使这些记忆里没有"数据库"这个词，语义搜索也能找到，因为它理解词与词之间的意思关系。用向量相似度实现。

为什么现在不用语义搜索？

三个原因。第一，规模小。当前记忆量是几十到几百条，关键词搜索够快，不需要更复杂的技术。第二，确定性。关键词搜索每次查询结果一样，语义搜索可能有微小变化。用户更信任稳定的结果。第三，零依赖。语义搜索需要 embedding 模型（把文字转成向量）和向量数据库（存和查向量），这些都是外部依赖。用户要额外安装配置，增加复杂度。

当记忆量达到万级（几千到几万条），关键词搜索性能下降，或者需要模糊匹配时，可以在原本的搜索方法上加上语义搜索。

二十、评分公式

```
score = meta_hits * 2.0 + body_hits + importance * 0.4 + use_count * 0.1
```

因素	权重	说明
meta_hits	2x	元数据匹配的词数
body_hits	1x	正文匹配的词数
importance	0.4	记忆的重要性标记
use_count	0.1	记忆被使用的次数
recency_boost	0.1-0.3	记忆的最近更新时间

长期记忆根据用户指示或者 llm 判断来加入。记忆更新发生在对话结束时、用户触发长期记忆保存时、后台定时扫描。记忆在 harness 启动或者会话恢复时加入 system_prompt。记忆加载有限制数量，最多 5 个文件。

八、记忆系统的 Harness 设计总结

Harness 保障	作用	实现位置
自动保存	每轮结束自动保存 Session Memory	query_engine.py <code>_update_session_memory</code>
长度限制	防止记忆文件太大撑爆上下文	session_memory <code>MAX_CHARS</code>
自动提取	LLM 自动判断并提取持久记忆	memory_extract <code>extract_memories_from_turn</code>
重复检测	内容签名防止重复记忆	schema <code>compute_memory_signature</code>
秘密检查	团队记忆不允许包含敏感信息	team <code>check_team_memory_secrets</code>
文件锁	防止并发写入冲突	manager <code>exclusive_file_lock</code>
备份机制	Auto-dream 整理前先备份	autodream <code>create_memory_backup</code>
权限隔离	整理任务只允许操作记忆目录	memory_extract <code>validate_tool_request</code>
后台执行	Auto-dream 不阻塞主对话	autodream <code>schedule_auto_dream</code>
回滚机制	整理失败可恢复	autodream <code>rollback_lock</code>

4. 任务恢复与会话持久化

中断恢复、多任务切换、后台任务（启动后台 agent 任务后，主会话退出）

会话恢复是从 Session Storage 恢复

存储结构（简单说）

```
~/openharness/data/sessions/  
├── my-project-a1b2c3d4/ ← 你的项目对应的文件夹  
│   ├── latest.json      ← 最新状态（每次对话都会更新这个）  
│   └── session-abc123.json ← 某个时间点的快照
```

JSON 文件里存了什么

```
{  
  "messages": [  
    ← ← ← 这就是完整的对话历史！  
    {"role": "user", "text": "帮我修复登录bug"},  
    {"role": "assistant", "text": "我来看看..."},  
    {"role": "user", "content": [{"type": "tool_result", "content": "找到问题了"}]},  
    {"role": "assistant", "text": "问题在 auth.py 第42行"}  
  ],  
  "tool_metadata": {  
    ← ← ← 运行时状态  
    "read_file_state": ["auth.py", "login.py"],  
    "invoked_skills": ["fix-bug"]  
  },  
  "model": "claude-sonnet-4-6",  
  "summary": "帮我修复登录bug"  
}
```

会话快照（Session Snapshot）：Agent 运行时状态序列化为 JSON 文件。包含完整消息历史（用户问题、AI 回复、工具调用记录）+ 10 维 tool_metadata（工具状态信息）。用户一次对话后（可能包含多次工具调用）保存一次/用户执行压缩命令时，不论是否成功或失败。

检查点（Checkpoint）：压缩检查点记录每次压缩前后的消息数/Token 数，便于压缩失败后回滚。

断点续传（Continuation）：has_pending_continuation() 检查最后消息是否含未消费的 ToolResult，即工具执行完成 Toolresult 已经返回，但 Ai 没有针对该内容生成回复。是则继续执行 AI 未回复的内容，否则等新输入。

状态恢复一致性：通过保留 tool_metadata (read_file_state、invoked_skills、async_agent_tasks、task_focus_state 等 10 个键) 保证恢复后 Agent 知道"之前做到哪了"。

本项目任务恢复流程：save_session_snapshot() 保存为 JSON 快照，load_session_snapshot() 恢复。恢复时先加载快照文件，清洗对话历史

清洗规则：

规则	说明
删除空 assistant 消息	is_effectively_empty() → 没有文字、没有 tool_use、没有图片
删除孤儿 tool_use	assistant 发了 tool_use，但后面没有对应的 tool_result → 删掉这条 assistant 消息
删除孤儿 tool_result	user 消息只有 tool_result，没有配对的 tool_use → 删掉这些 tool_result，保留其他文字内容

（三种情况，删除空的 assistant 消息，即工具调用执行完成，但没来得及调用 model 处理结果时中断、删孤儿 tool_use/tool_result，即工具调用请求和工具调用结果不配对），构建 queryengine，加载恢复数据。再检测待处理状态（假如最后一条消息是 tool_result，即中断在工具执行后还没有调用模型，那就可以直接调用模型处理 tool_result），启动和用户的交互

追问应对：「恢复后真能无缝继续吗？」——只要 tool_metadata 完整，Agent 知道"读了哪些文件、加载了哪些技能、当前任务是什么"。但如果外部状态变了（文件被人改过、进程被终止），需要 Agent 自己发现和适应。

Harness 保障	说明	实现方式
自动保存	每轮结束自动保存	runtime.py 中 save_snapshot
原子写入	防止写入中断导致损坏	atomic_write_text
双重保存	latest + session-id	确保至少一份可用
Sanitization	恢复时清理无效消息	sanitize_conversation_messages
待处理检测	恢复后自动继续中断点	has_pending_continuation
文件隔离	不同项目分开存储	项目路径 hash 命名
摘要生成	快照包含会话摘要	首条用户消息截断
索引维护	支持会话列表显示	list_session_snapshots
Token 统计	恢复后继续统计 usage	UsageSnapshot 保存恢复
Session Memory 同步	恢复时同步 Session Memory	prepare_session_memory_metadata

其中最重要的就是原子写入，发生在每 loop 后的快照保存时，先写草稿，写完后进行 replace

5. 工具系统 2.工具基类

...

步骤 1: Pre-tool-use hook (工具执行前的钩子)+工具查找

步骤 2: 输入验证 (参数是否正确)

步骤 3: 权限检查 (能不能执行)

步骤 4: 执行工具 (实际操作)

步骤 5: 输出截断 (结果太长就截断), 把完整内容存到临时文件, 只给 AI 看截断版本 + 文件路径。

步骤 6: Post-tool-use hook (工具执行后的钩子)

步骤 7: carryover 记录 (记录执行状态) 到 tool_metadata

...

设计思想: 所有工具继承 BaseTool, 统一接口

```
# tools/base.py
class BaseTool(ABC):
    name: str # 工具名
    description: str # 描述
    input_model: type[BaseModel] # 输入验证
    @abstractmethod
    async def execute(self, arguments, context) -> ToolResult:
        """每个工具实现这个方法"""
    def is_read_only(self, arguments) -> bool:
        """声明是否只读(影响权限判断)"""
        return False
```

ToolExecutionContext - 工具共享上下文:

LLM 调用不存在工具的处理流程:

第一步: LLM 发起工具调用

LLM 在响应中返回一个 tool_use 块, 包含工具名称、参数和唯一 ID。比如 LLM 想删除文件, 返回了"delete_file"这个工具调用请求。

第二步: 系统查找工具定义

QueryEngine 的 _execute_tool_calls 函数遍历所有 tool_use, 对每个工具调用, 它会去 ToolRegistry 中查找对应的工具定义。**ToolRegistry 是一个工具注册表, 存储了所有可用工具的定义, 类似字典结构。**

第三步: 发现工具不存在

当调用 tool_registry.get("delete_file")时, 返回了 None, 表示 Registry 中没有这个工具。这可能是因为 LLM 幻觉编造了工具名, 或者工具被移除, 或者拼写错误。

第四步: 构建错误结果

系统不会忽略这个调用, 而是构建一个 ToolResultBlock 作为返回结果。这个结果块包含三个关键信息: tool_use_id 对应刚才的工具调用 ID, content 是错误信息"Unknown tool: delete_file", is_error 标记为 True 表示这是个错误。

第五步: 错误结果返回给 LLM

这个错误结果作为 tool_result 返回给 LLM, 放在下一轮对话的 user 角色消息中。LLM 看到的结果明确告诉它: 你调用的 delete_file 工具不存在。

第六步: LLM 调整策略

LLM 收到错误反馈后，理解工具不存在的事实。它会分析错误信息中提供的可用工具列表，然后调整策略。比如 LLM 可能会说"抱歉，`delete_file` 不存在，我改用 `bash` 工具执行删除命令"，然后在下一轮返回正确的工具调用 `bash`。

整个流程的核心设计原则是：给 LLM 明确的错误反馈，而不是静默失败。如果直接忽略，LLM 会一直等待结果卡住；如果返回空结果，LLM 会误以为成功继续错误操作；只有返回明确的错误，LLM 才能知道发生了什么并自我修正

工具调用完整流程

LLM 首先在回复里告诉我们它想用什么工具。比如 LLM 说我要用 `read_file` 工具读取 `main.py`，它会返回一个 `tool_use` 块，里面有工具名称、参数和唯一编号。

系统收到这个请求后，会先去看看这个工具存不存在。我们有个工具注册表，里面记录了所有可用的工具。如果 LLM 请求的工具不在表里，我们就直接告诉 LLM 说这个工具不存在，LLM 看到错误信息后会自己调整策略，比如换个工具或者放弃这个操作。

工具找到后，接下来要做参数检查。因为每个工具都有自己的参数要求，比如 `read_file` 需要 `path` 参数，`bash` 需要 `command` 参数。我们用 `Pydantic` 这个库来验证参数，如果 LLM 传的参数不对，比如漏了必填参数或者类型错了，我们会把具体的错误信息返回给 LLM，LLM 就能知道哪里错了，下次调用时会修正。

参数没问题后，就可以执行工具了。不同工具有不同的执行逻辑。`bash` 工具会启动一个子进程去跑 `shell` 命令，`read_file` 工具会读取文件内容并加上行号格式化，`agent` 工具会创建一个子 `Agent` 进程。每个工具执行完都会返回一个结果，包含输出内容和一个错误标记。

拿到结果后，我们把它包装成 `tool_result` 格式返回给 LLM。`tool_result` 里有个关键字段叫 `tool_use_id`，它要对应之前 `tool_use` 的编号，这样 LLM 就知道这个结果是对应哪个工具调用的。

最后把这个结果加到对话历史里，LLM 在下一轮对话时会看到这个结果，然后根据结果决定下一步做什么。如果结果有错误，LLM 可能会重试或者换个方法；如果成功，LLM 继续后续操作。

这里有个设计细节值得一提。LLM 可能一次请求执行多个工具，比如同时读三个文件。我们会用并行方式执行这些工具，这样速度快，用户不用等太久。但并行执行有个问题，万一其中一个失败了怎么办？我们用 `asyncio.gather` 加 `return_exceptions` 参数来处理，确保单个工具失败不会影响其他工具，每个工具的结果都会返回，LLM 能清楚看到哪些成功哪些失败。

步骤 3 的工具权限检查：

权限检查不是简单判断，而是六个关卡按顺序检查。

关卡一：敏感文件硬保护（最严格）

这是第一关，检查 LLM 要操作的文件是不是敏感文件。什么是敏感文件？比如 `SSH` 私钥、`AWS` 密钥、环境变量文件。这些文件特别重要，如果 LLM 读到或者改了，你的密钥可能泄露。所以系统有个黑名单，不管什么情况，黑名单里的文件都不能碰。

这个关卡的特点是"硬"，硬到什么程度？**用户不能改这个黑名单，不能绕过**，即使你切换到全自动模式，即使你配置了允许规则，敏感文件还是不能碰。这是系统的安全底线。

关卡二：工具黑名单

这是第二关，检查 LLM 用的工具是不是被禁用了。**用户可以配置一个黑名单，比如禁用 `agent` 工具，不让 LLM 创建子 `Agent`。**如果 LLM 想用这个工具，第二关直接拦住。这个关卡的作用是**让用户禁用某些危险工具**。比如你觉得创建子 `Agent` 太复杂不想用，就把它加到黑名单，LLM 就不能用了。

举例：用户配置 `denied_tools=["agent"]`，LLM 想调用 `agent` 工具创建子 Agent，系统第二关检查发现 `agent` 在黑名单，直接拒绝，告诉 LLM"这个工具被禁用了"。

关卡三：工具白名单

这是第三关，检查 LLM 用的工具是不是在信任名单里。用户可以配置一个白名单，比如信任 `read_file` 工具，允许 LLM 自由读取文件。如果工具在白名单，这关直接放行，后面的关卡不用检查了。白名单的作用是让用户信任某些工具，跳过后续检查。比如你觉得读取文件很安全，就把 `read_file` 加到白名单，LLM 读文件时不会触发确认。举例：用户配置 `allowed_tools=["read_file"]`，LLM 想读取某个文件，系统第三关检查发现 `read_file` 在白名单，直接允许，不会问用户确认。

注意：白名单不能绕过硬保护。即使 `read_file` 在白名单，LLM 想读取敏感文件还是会被第一关拦住。因为第一关在白名单前面。

关卡四：路径规则

这是第四关，检查 LLM 操作的文件路径是否符合规则。用户可以配置路径规则，比如允许访问 `src` 目录，禁止访问 `config` 目录。路径规则用通配符匹配，可以精细控制。如果文件路径匹配了禁止规则，这关直接拦住。如果匹配了允许规则，继续检查后面的关卡（不立即放行，因为后面可能有其他限制）。

举例：用户配置 `path_rules=["config/*禁止"]`，LLM 想修改 `config/settings.json`，系统第四关检查发现路径匹配禁止规则，直接拒绝，告诉 LLM"这个路径被禁止访问"。

关卡五：命令黑名单

这是第五关，专门针对 `bash` 工具，检查 LLM 执行的命令是不是危险命令。用户可以配置命令黑名单，比如禁止 `"rm -rf"` 强制删除命令。如果 LLM 想执行这个命令，这关直接拦住。命令黑名单的作用是防止 LLM 执行危险 shell 命令。有些命令破坏力太大，比如强制删除整个目录，用户希望禁止。

举例：用户配置 `denied_commands=["rm -rf"]`，LLM 想执行 `"rm -rf /tmp/"`，系统第五关检查发现命令匹配黑名单，直接拒绝。

关卡六：模式判断（最后的兜底）

这是最后一关，根据当前的权限模式决定结果。有三种模式，我后面详细讲。简单说：

全自动模式：直接放行，不需要用户确认；规划模式：只读操作放行，修改操作直接拒绝；

默认模式：只读操作放行，修改操作需要用户确认

这关是最后的判断，决定操作是自动执行、直接拒绝还是先问问用户。

六、如何判断工具是读还是写

每个工具有个 `is_read_only` 方法。

比如 `FileReadTool`（在 `file_read_tool.py:27-29`）：

```
def is_read_only(self, arguments: FileReadToolInput) -> bool:
    return True # 读文件是只读操作
```

`BashTool` 没有定义这个方法，所以默认返回 `False`（在 `base.py:46-49`）：

```
def is_read_only(self, arguments: BaseModel) -> bool:
    del arguments
    return False # 默认是写操作
```

内容一：工具输出截断和外溢

工具执行完返回结果，这个结果可能特别长。比如你让工具读取一个一万行的日志文件，输出可能有几十万字符。如果这个输出直接塞到对话历史里，上下文瞬间就爆了，后面的对话就没法继续。

所以我们有个截断机制。系统设定了一个阈值，叫 `inline_limit`，默认是 12000 字符。如果工具输出超过这个阈值，我们就截断它，不把完整内容放进对话历史。将完整输出保存到一个临时文件里。这个文件放在专门的 `artifact` 目录下，文件名包含时间戳、工具名称和随机 ID，方便后续查找。同时我们在对话历史里放一个摘要，告诉 LLM 输出被截断了，完整内容在哪个文件里，还附带一个开头部分的预览，让 LLM 大概知道输出是什么内容。

这个设计有个关键问题：截断阈值怎么选？太小了会频繁截断，LLM 看不到完整信息；太大了上下文容易爆。我们选 12000 字符是经过权衡的，大概相当于一页代码的量，足够 LLM 理解上下文，又不会占用太多空间。

外溢文件怎么管理呢？文件生成后记录到 `active_artifacts` 列表里。这个列表是 `tool_metadata` 的一部分，压缩恢复时会用到。当对话压缩时，系统知道有哪些外溢文件，可以在必要时引用它们。文件本身不会自动删除，会保留在 `artifact` 目录里，用户可以手动查看。

压缩时怎么处理外溢内容呢？压缩生成的摘要里会提到有外溢文件存在，告诉恢复后的 LLM 如果需要完整内容可以去哪个文件找。这样即使对话历史被压缩了，关键信息也不会丢失

工具执行超时处理

有些工具执行时间可能很长，比如 `bash` 工具跑一个编译命令，可能要几分钟。但也不能无限等待，万一命令卡住了，整个 `Agent Loop` 就阻塞了，所以每个工具都有超时设置。

`bash` 工具默认超时 600 秒，也就是 10 分钟。超过这个时间，系统会强制终止进程。系统先尝试优雅终止，发送 `SIGTERM` 信号让进程自己退出。如果进程 2 秒内不退出，就强制杀死，发送 `SIGKILL` 信号。进程被终止后，系统会收集已经产生的输出，即使命令没跑完，也可能有部分结果。这些部分输出返回给 LLM，同时告诉 LLM 命令超时了。LLM 看到超时信息，可以决定是重试、调整命令参数，还是放弃这个操作换别的方案。

超时阈值怎么选择呢？

`bash` 工具给了用户自定义的空间。参数里有 `timeout_seconds` 字段，用户可以指定超时时间，范围是 1 到 600 秒。如果用户知道某个命令会跑很久，比如一个大项目的编译，可以设置更长的超时。默认 600 秒是平衡大多数场景的选择，普通命令几秒就完了，复杂命令也不会太离谱。

部分输出怎么处理呢？

超时那一刻，进程可能已经输出了一些内容。系统用 `_drain_available_output` 函数读取已有的输出，即使进程还在跑，先把能读到的内容拿出来。这些内容格式化后返回，加上超时提示，让 LLM 知道命令跑了一半被中断了，也能看到已经产生的结果。

用户怎么干预超时设置呢？

通过工具参数。LLM 在调用 `bash` 工具时，可以传入 `timeout_seconds` 参数。比如 LLM 知道要跑一个耗时测试，可以设置 `timeout_seconds=300`。用户也可以在对话中告诉 LLM 说这个命令需要更长时间，LLM 下次调用时会调整超时参数。

工具错误分类和恢复

工具执行失败有多种原因，每种原因有不同的处理方式。理解这些错误分类，能帮助设计更健壮的系统。

第一类是工具不存在。LLM 请求的工具名称在 Registry 里找不到。这种情况返回 "Unknown tool: xxx" 错误，LLM 看到后知道工具不存在，可以调整策略换其他工具。

第二类是参数无效。Pydantic 验证失败，比如必填参数缺失、类型错误、范围超出。返回具体验证错误信息，LLM 知道哪个参数有问题，下次调用可以修正。

第三类是权限拒绝。权限检查不通过，分为硬拒绝和需要确认两种。硬拒绝返回拒绝原因，LLM 知道这个操作不允许。需要确认但用户拒绝，同样返回拒绝信息。

第四类是执行异常。工具 execute 方法抛出异常，比如文件不存在、进程启动失败、网络请求失败。返回异常信息，LLM 知道发生了什么错误。

第五类是超时。工具执行超过设定时间被强制终止。返回部分输出加超时提示，LLM 知道命令跑了一半被中断。

第六类是输出截断。工具输出太长被截断，完整内容保存到文件。返回截断摘要，LLM 知道完整内容在哪里可以找到。

错误信息格式是什么样的呢？

所有错误都通过 ToolResult 返回，is_error 标记为 True，output 包含错误描述。错误描述尽量具体，让 LLM 理解发生了什么。比如参数错误会列出具体的哪个参数有问题，权限拒绝会说明拒绝原因，执行异常会包含异常消息。

LLM 如何理解错误呢？

LLM 看到 is_error=True 的结果，知道这次调用失败了。然后分析 output 里的错误信息，理解失败原因。LLM 有能力根据错误调整策略，比如参数错误就修正参数重试，权限拒绝就换别的方案，工具不存在就用其他工具替代。

重试策略是什么呢？

LLM 自己决定是否重试。系统不会自动重试，因为有些错误重试也没用，比如权限拒绝。

LLM 根据错误类型判断：参数错误可以修正后重试，网络错误可以重试，权限拒绝就不能重试只能换方案。

当 LLM 一次请求两个 write_file 工具写同一个文件时：并行执行，两个写入操作都会完成，但后完成的会覆盖先完成的的内容。用户最终看到的文件内容是第二个工具写入的内容，第一个工具写入的内容丢失。为什么没有锁机制呢？设计上做了权衡。加锁需要知道哪些操作会冲突，这需要额外的检测逻辑。而且锁会降低并发性能，原本并行执行的操作变成串行。大多数情况下，LLM 不会请求冲突操作，加锁的收益不大。如果 LLM 真的请求了冲突操作怎么办？依赖 LLM 的自我修正。LLM 在下一轮看到两个工具结果，可能发现写入的内容不对，然后理解发生了冲突，再修正文件内容。系统层面不干预，因为不知道哪个写入是正确的。

问题：

递归工具调用未处理，没有限制最大递归深度

工具资源消耗，即 AI 连续调用大量 bash 命令，或者启动很多 agent 子进程，可能耗尽系统资源只有简单处理：bash 有 timeout 上限、agent 子进程会被追钟，但没有限制并发数量。

恶意命令注入有处理：配置命令黑名单

敏感信息泄露：敏感路径禁止访问

工具输出中的敏感信息，即工具执行合法操作，但输出包含敏感信息未处理。

工具系统的 harness 就是在 AI 和真实执行之间加了七道安检门：

- 1. 钩子拦截
- 2. 工具不存在拦截
- 3. 参数格式错误拦截
- 4. 权限拒绝拦截
- 5. 执行
- 6. 输出截断
- 7. 记录 + 后钩子

任何一道门说“不行”，工具就不会跑。这就是 OpenHarness 能安全让 AI 操作你电脑的原因。

六、总结：Harness 处理的情况一览

风险	是否处理	处理方式	代码位置
输出过大	✓	两层截断 + 外溢到文件	bash_tool.py:139 , query.py:524
执行超时	✓	timeout + 强制杀进程 + 部分输出	bash_tool.py:60
参数错误	✓	Pydantic 校验 + 错误返回	query.py:916
权限拒绝	✓	三层权限检查 + 用户确认	checker.py:75
工具异常	✓	转成 ToolResult + 不中断对话	query.py:827
并发写冲突	⚠ 部分	有文件锁但工具未使用	file_lock.py
递归调用	⚠ 部分	有记录但无深度限制	query.py:286
资源耗尽	⚠ 部分	有追踪但无并发限制	query.py:341
恶意命令	✓	denied_commands 配置	checker.py:119
敏感路径	✓	硬编码禁止名单	checker.py:18
输出敏感信息	✗	未处理	潜在改进点
交互式命令	✓	执行前检测 + 拒绝	bash_tool.py:155
调用频率	✓	max_turns 限制回合数	cli 参数

这就是工具系统的完整 Harness 设计——每一层都是一道安全门，但仍有几个潜在改进点值得注意。

6. 钩子

事件触发时机

事件	触发时机	典型用途
SESSION_START	启动 OpenHarness 时	初始化环境、加载配置
SESSION_END	关闭 OpenHarness 时	清理临时文件、保存日志
PRE_TOOL_USE	执行工具之前	检查是否允许、记录意图
POST_TOOL_USE	执行工具之后	记录结果、自动通知
PRE_COMPACT	压缩对话之前	保存压缩前的状态
POST_COMPACT	压缩对话之后	记录压缩结果
USER_PROMPT_SUBMIT	用户提交消息时	检查消息内容、记录输入
STOP	会话停止时	清理资源
SUBAGENT_STOP	Worker 完成时	记录 Worker 结果

四种钩子类型：

command 钩子：执行 shell 命令，比如检查文件、记录日志、发送通知；

http 钩子：向外部服务发送通知，比如 Slack、webhook。

prompt 钩子：让 LLM 判断是否允许某个操作

Agent 钩子：LLM 深度思考判断

十三、Harness 总结表

Harness	作用	代码位置
timeout_seconds	防止钩子卡住	schemas.py:15
block_on_failure	钩子失败后是否阻止	schemas.py:17
matcher	精准匹配特定工具/事件	executor.py:215
priority	控制钩子执行顺序	loader.py:27
环境变量注入	钩子脚本获取上下文	executor.py:94
\$ARGUMENTS 替换	钩子获取完整 payload	executor.py:223
结果聚合	多钩子协作，一个阻止就全部阻止	types.py:28
热重载	修改配置立即生效	hot_reload.py:19
子进程隔离	Command 钩子在独立进程	executor.py:88

钩子系统的 Harness 就是：

1. 超时保护：防止钩子卡住
2. block 控制：用户决定钩子失败后的行为，可以继续或者阻止工具运行
3. 精准匹配：钩子可以只匹配特定工具/事件
4. 执行顺序：priority 控制优先级，同一事件的多个钩子，按 priority 排序
5. 上下文注入：环境变量 + \$ARGUMENTS

6. 热重载：修改配置立即生效

7. 权限系统

一、权限系统是什么，为什么需要

权限系统简单说就是：控制 LLM 能做什么操作的安全关卡。

想想这个场景：你让 LLM 帮你写代码，LLM 说要读取你的配置文件，然后修改，最后还要执行一个删除命令。这些操作可能改你的代码、删你的文件、甚至泄露你的密钥。如果没有权限控制，LLM 想做什么就做什么，太危险了。

所以我们需要一个关卡，在 LLM 执行操作前先检查：这个操作能做吗？是不是危险操作？需要用户同意吗？

权限系统就是这个关卡。每次 LLM 想用工具前，系统先问权限系统“能做吗”，权限系统回答“可以”、“不行”或者“先问问用户”。

二、权限检查的顺序（六个关卡）

权限检查不是简单判断，而是六个关卡按顺序检查。

关卡一：敏感文件硬保护（最严格）

这是第一关，检查 LLM 要操作的文件是不是敏感文件。什么是敏感文件？存密钥、凭证、密码的文件。比如 SSH 私钥、AWS 密钥、环境变量文件。这些文件特别重要，如果 LLM 读到或者改了，你的密钥可能泄露。所以系统有个黑名单，不管什么情况，黑名单里的文件都不能碰。

这个关卡的特点是“硬”，硬到什么程度？**用户不能改这个黑名单，不能绕过**，即使你切换到全自动模式，即使你配置了允许规则，敏感文件还是不能碰。这是系统的安全底线。

关卡二：工具黑名单

这是第二关，检查 LLM 用的工具是不是被禁用了。**用户可以配置一个黑名单，比如禁用 agent 工具，不让 LLM 创建子 Agent**。如果 LLM 想用这个工具，第二关直接拦住。这个关卡的作用是**让用户禁用某些危险工具**。比如你觉得创建子 Agent 太复杂不想用，就把它加到黑名单，LLM 就不能用了。

举例：用户配置 `denied_tools=["agent"]`，LLM 想调用 agent 工具创建子 Agent，系统第二关检查发现 agent 在黑名单，直接拒绝，告诉 LLM“这个工具被禁用了”。

关卡三：工具白名单

这是第三关，检查 LLM 用的工具是不是在信任名单里。**用户可以配置一个白名单，比如信任 read_file 工具，允许 LLM 自由读取文件**。如果工具在白名单，这关直接放行，后面的关卡不用检查了。白名单的作用是让用户信任某些工具，跳过后续检查。比如你觉得读取文件很安全，就把 read_file 加到白名单，LLM 读文件时不会触发确认。举例：用户配置 `allowed_tools=["read_file"]`，LLM 想读取某个文件，系统第三关检查发现 read_file 在白名单，直接允许，不会问用户确认。

注意：白名单不能绕过硬保护。即使 read_file 在白名单，LLM 想读取敏感文件还是会被第一关拦住。因为第一关在白名单前面。

关卡四：路径规则

这是第四关，检查 LLM 操作的文件路径是否符合规则。**用户可以配置路径规则，比如允许访问 src 目录，禁止访问 config 目录**。路径规则用通配符匹配，可以精细控制。如果文件路

径匹配了禁止规则，这关直接拦住。如果匹配了允许规则，继续检查后面的关卡（不立即放行，因为后面可能有其他限制）。

举例：用户配置 `path_rules=["config/*禁止"]`，LLM 想修改 `config/settings.json`，系统第四关检查发现路径匹配禁止规则，直接拒绝，告诉 LLM"这个路径被禁止访问"。

关卡五：命令黑名单

这是第五关，专门针对 `bash` 工具，检查 LLM 执行的命令是不是危险命令。用户可以配置命令黑名单，比如禁止 `"rm -rf"` 强制删除命令。如果 LLM 想执行这个命令，这关直接拦住。命令黑名单的作用是防止 LLM 执行危险 shell 命令。有些命令破坏力太大，比如强制删除整个目录，用户希望禁止。

举例：用户配置 `denied_commands=["rm -rf"]`，LLM 想执行 `"rm -rf /tmp/"`，系统第五关检查发现命令匹配黑名单，直接拒绝。

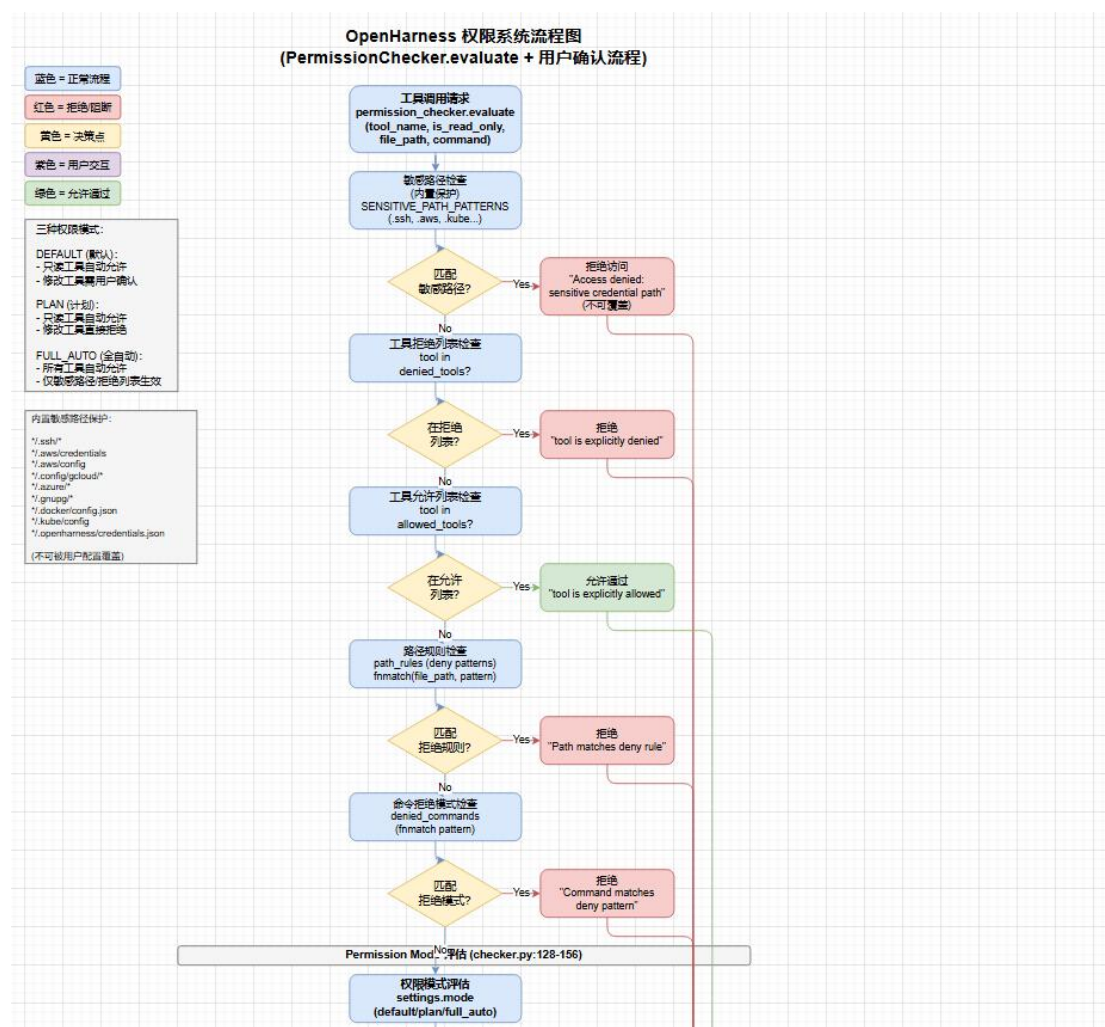
关卡六：模式判断（最后的兜底）

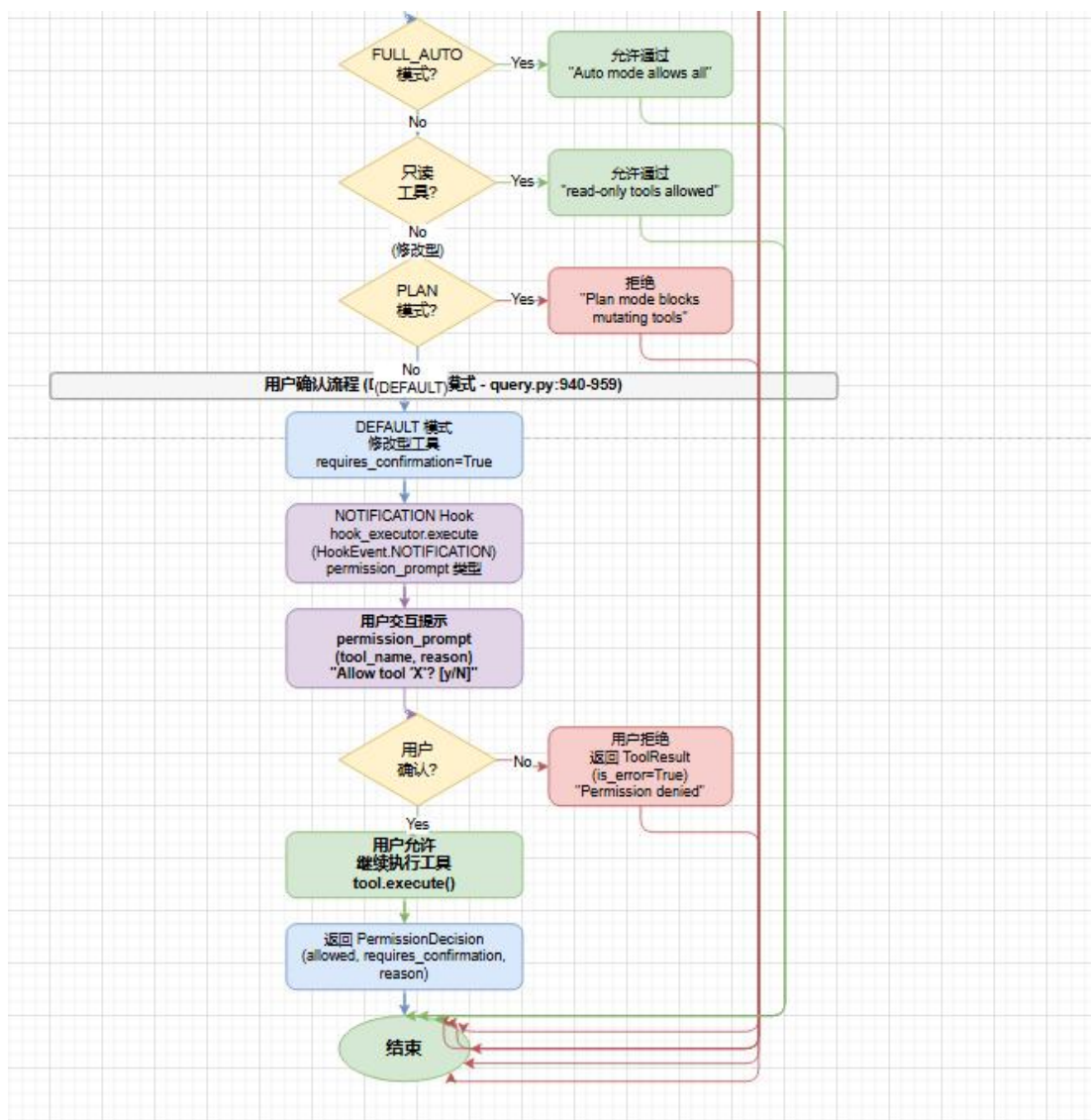
这是最后一关，根据当前的权限模式决定结果。有三种模式，我后面详细讲。简单说：

全自动模式：直接放行，不需要用户确认；规划模式：只读操作放行，修改操作直接拒绝；

默认模式：只读操作放行，修改操作需要用户确认

这关是最后的判断，决定操作是自动执行、直接拒绝还是先问问用户。





三、权限检查顺序的设计逻辑

顺序的原则是：最严格的在最前面，最宽松的在最后面。

第一关是硬保护，最严格，因为涉及密钥安全，绝对不能放松。

第二关是工具黑名单，用户明确禁用的工具，也应该早拦。

第三关是工具白名单，用户明确信任的工具，可以早放。

第四关是路径规则，文件级别的控制，比模式更具体。

第五关是命令黑名单，针对 shell 命令的危险控制。

第六关是模式判断，最宽松，根据模式做最后决定。

这样设计的好处：危险操作在前面关卡就拦住了，不会到最后关才被发现。用户配置的规则优先级高于模式，配置可控。硬保护始终有效，不会被任何规则绕过。

四、三种权限模式详解

默认模式：这是最常用的模式，平衡了效率和安全性。规则很简单：只读操作自动执行，修改操作要问用户。只读操作就是不会改变任何东西的操作，比如读取文件、搜索文件、列出目录。这些操作没有风险，LLM 想做就做，不用问用户。修改操作就是会改变东西的操作，比如写文件、删文件、执行命令。这些操作有风险，可能改代码、删数据。所以系统暂停，

弹对话框问用户"LLM 想做这个操作，你同意吗"。用户点同意就执行，点拒绝就不执行。

规划模式：这是最严格的模式，禁止所有修改操作。规则是：只读操作自动执行，修改操作直接拒绝，不问用户。规划模式的目的是让 LLM 只做分析和规划，不动手执行。你让 LLM 分析问题、设计方案时，用这个模式防止 LLM 忍不住改代码。规划模式适合代码评审、问题分析、方案设计，需要 LLM 思考但不动手。

全自动模式：这是最宽松的模式，允许所有操作自动执行。规则是：所有操作都自动执行，不问用户。全自动模式的目的是让 LLM 自主完成任务，最大化效率。你信任 LLM，或者任务紧急需要快速执行时，用这个模式。但注意：全自动模式也不能绕过硬保护。即使全自动模式，LLM 想改敏感文件还是会被第一关拦住。硬保护是绝对的。全自动模式适合自动化任务、CI/CD 流程、完全信任 LLM 的场景。

五、路径规则详解

路径规则是文件级别的访问控制，让用户精细配置哪些目录可以访问，哪些不能。

路径规则的配置：用户在配置文件里写规则列表，每个规则有两部分：模式和动作。

模式是文件路径的通配符。星号匹配任意字符，比如"src/*"匹配 src 目录下所有文件。双星号可以匹配多层目录，比如"src/**"匹配 src 及其子目录所有文件。

动作是允许或禁止。允许表示匹配的路径可以访问，禁止表示匹配的路径不能访问。

路径规则的检查方式：系统拿到文件路径后，按顺序检查规则列表。如果路径匹配禁止规则，直接拒绝。如果匹配允许规则，继续检查后面的关卡（因为后面可能还有其他限制）。

检查顺序是配置顺序，先配置的规则先检查。所以如果你配置了"允许 src"然后"禁止 src/secrets"，src/secrets 的文件会被禁止，因为它匹配了第二条规则。

路径规则和模式的关系：路径规则独立于模式。即使全自动模式，路径规则禁止的文件也不能访问。路径规则在模式检查前面，如果路径规则拒绝了，就不会进入模式判断。

六、敏感路径硬保护详解

硬保护是系统内置的安全底线，保护密钥和凭证文件。这些文件如果被 LLM 读取或修改，你的云服务密钥、SSH 密钥可能泄露，后果严重。

硬保护为什么不能绕过：硬保护是系统代码内置的，不是用户配置的。用户不能修改黑名单内容，不能添加例外规则，不能通过切换模式绕过。

为什么这样设计？发生过真实事故。LLM 被恶意 prompt 诱导，读取 AWS 密钥然后发送到外部服务器，导致密钥泄露。硬保护是防止这种情况的最后一道防线。

即使用户配置了路径规则允许访问.aws 目录，即使切换到全自动模式，即使工具在白名单里，LLM 想读取.aws/credentials 时，第一关硬保护直接拦住。

硬保护的边界情况

如果用户真的需要 LLM 操作敏感文件怎么办？当前设计不支持，必须用户手动操作。这是安全设计的有意选择：敏感文件的访问权不给 LLM，只给用户。

七、交互式确认详解

当权限检查决定需要用户确认时，系统会弹出对话框让你决定。

默认模式下，LLM 执行修改操作时触发确认。具体是：第六关模式判断发现是修改操作，返回"需要确认"，系统暂停工具执行，弹出对话框。对话框显示三个信息：工具名称、操作内容、确认原因。工具名称告诉你是哪个工具，比如"write_file"。操作内容显示具体做什么，比如"修改 main.py"。确认原因解释为什么需要确认，比如"默认模式下修改操作需要用户确认"。你看清楚后决定点同意还是拒绝。

多工具并行时怎么确认

LLM 可能一次请求执行多个工具，比如同时读三个文件、写两个文件。其中写文件需要确认，怎么处理？系统用**锁机制串行化确认过程**。一个确认对话框显示时，其他工具的确认请求排队等待。你处理完第一个确认，系统再显示第二个确认。

你同意了一次写文件，后续写文件还会再弹出确认吗？会。每次工具调用独立检查，不缓存确认结果。

八、其他补充知识

权限模式如何切换？

用户通过命令切换模式，比如输入 `"/permissions full_auto"` 切换到全自动模式，输入 `"/permissions plan"` 切换到规划模式，输入 `"/permissions default"` 切换回默认模式。模式切换后立即生效，后续工具调用按新模式判断。

只读判断由工具自己定义？

系统怎么知道一个操作是不是只读？每个工具有个 `is_read_only` 方法，工具自己告诉系统。`read_file` 工具的 `is_read_only` 返回 `True`，因为读取不修改文件。`write_file` 工具的 `is_read_only` 返回 `False`，因为写入修改文件。`bash` 工具的 `is_read_only` 要看命令内容，有些命令只读比如 `"ls"`，有些命令修改比如 `"rm"`。

权限配置存储在哪儿？

权限配置存储在 `settings.json` 文件，包括：当前模式、工具白名单、工具黑名单、路径规则、命令黑名单？

用户可以手动编辑配置文件，也可以通过命令修改配置。

8. agent 协作与通信

Coordinator 不能执行普通工具操作，比如 `read_file`、`write_file`、`bash`。**Coordinator** 只能用三个专属工具：**agent**、**send_message**、**task_stop**。

原因一：专注协调

Coordinator 的核心任务是协调多个 **Worker**。如果 **Coordinator** 自己也读文件、写文件，会分散注意力。**Coordinator** 应该站在全局视角思考：谁应该做什么、结果如何综合、下一步怎么做。

原因二：避免冲突

Coordinator 和 **Worker** 可能操作同一个文件。如果 **Coordinator** 也在写文件，**Worker** 也在写文件，会产生冲突。**Coordinator** 不执行操作，就避免了这种冲突。

原因三：明确责任

Coordinator 负责决策，**Worker** 负责执行。责任清晰，容易追踪。如果 **Coordinator** 也执行操作，出问题时很难判断是决策错还是执行错。

两种组织模式：

模式一：Boss-Worker（主从模式）

这是最常用的模式。一个 **Coordinator** (Boss) 管理多个 **Worker**。**Coordinator** 分配任务，**Worker** 执行任务，完成后汇报。

创建时机：任务开始前预先规划；创建者：只有 Coordinator 能创建 Worker；适用场景：

分工明确的复杂任务。出现问题后可以返回结果，让 Coordinator 决定后续处理，比如继续使用当前 worker 处理或者创建新 worker

模式二：Subagent Spawning（子代理生成）

这是最灵活的模式。主 Agent 在需要时动态创建子 Agent 处理独立子任务。子 Agent 完成后自动销毁。**创建时机：任务执行中动态决定；创建者：任何 Agent 都能创建子 Agent**（理论设计）；适用场景：执行中发现新问题需要处理。

Coordinator 的设计逻辑

环境变量：CLAUDE_CODE_COORDINATOR_MODE，当这个变量设置为"1"、"true"或"yes"时，系统判定当前 Agent 是 Coordinator。

只能用三个专属工具，不能用普通工具。这三个工具是 Coordinator 管理 Worker 的手段：agent 工具创建 worker（Worker 每次启动都是新进程，没有历史上下文。Coordinator 的 prompt 必须像给陌生人写信一样，把所有必要信息都写清楚）、send_message 向 worker 发送新消息、task_stop 停止 worker。

Coordinator 的系统提示词告诉 Coordinator 如何扮演协调者角色。这个提示词很长，包含角色定义、工具说明、Worker prompt 写法、并发策略等。

Worker完成任务后返回XML格式的task-notification。Coordinator要识别、解析、理解这个结果。

task-notification的完整格式

```
<task-notification>
<task-id>worker-1@my-team</task-id>
<status>completed</status>
<summary>Agent “研究认证代码” completed</summary>
<result>
分析完成：
- validate.ts:42 检查 session_id
- session.ts 管理 session生命周期
- types.ts 定义Session类型
建议修改validate.ts添加JWT验证
</result>
<usage>
  <total_tokens>5000</total_tokens>
  <tool_uses>12</tool_uses>
  <duration_ms>30000</duration_ms>
</usage>
</task-notification>
```

各字段含义：

- task-id: Worker的ID，Coordinator用这个ID继续或停止Worker
- status: 完成状态，completed（成功）、failed（失败）、killed（被停止）
- summary: 简短描述，给Coordinator快速了解结果
- result: 详细结果内容，Worker的分析报告或最终输出
- usage: 资源消耗，tokens数量、工具调用次数、执行时间

worker 简介：

送给 worker 的 prompt 必须详细且自包含。Worker 是新进程，没有历史上下文。Coordinator 要把所有必要信息写进 prompt。

Worker 进程独立运行。有自己的内存空间、自己的工具集、自己的 Agent Loop。不会和 Coordinator 互相干扰。Worker 的上下文独立于 Coordinator，双方只通过 stdin/ 邮件传递任

务。Coordinator 和 Worker 通过 stdin 管道通信，JSON 格式确保准确传递。
环境变量区分角色。Coordinator 有标记，Worker 没有标记。双方知道自己的角色定位。

内置 Agent 类型

在 `agent_definitions.py:510-620`:

类型	角色	工具权限	用途
general-purpose	全能	全部工具	复杂任务、多步骤搜索
Explore	搜索	只读工具	快速搜索代码，不能改文件
Plan	设计	只读工具	设计实现方案，不能改文件
worker	实现	全部工具	写代码、修 bug
verification	验证	只读 + 测试	验证实现是否正确
claude-code-guide	文档	只读 + WebFetch	回答 Claude Code 使用问题

相关 harness:

1. Agnet 定义隔离

每个 Agent 类型有独立的工具权限配置，Explore、Plan、verification: 禁用写工具，只能读；
worker: 全部工具可用

2. 子进程隔离

Worker 是独立进程，有自己的内存、上下文

Worker 崩溃不影响 Coordinator

Worker 不能直接访问 Coordinator 的变量

3. 任务追踪

Coordinator 每次启动 Worker，都记录到 `tool_metadata`

每个 Worker 的状态被追踪，`notification_sent=False` 表示 Coordinator 还没收到结果，
Coordinator 回合结束后，系统轮询检查

4. 自动等待通知

Coordinator 回合结束后，系统自动等待 Worker 完成

检查 `tool_metadata["async_agent_tasks"]`，若有未通知的 Worker 就等待；Worker 完成后，格式化成 XML，作为"用户消息"发给 Coordinator

Harness 作用：确保 Coordinator 在 Worker 完成后能收到通知，不会遗漏

5. worker 崩溃重启

Coordinator 发后续指令时，检查进程是否活着，死了就自动重启，新进程收到后续指令，继续执行

6. 防止 worker 递归启动子 worker

7. worker 有最大回合数限制

Harness	作用	代码位置
Agent 定义隔离	不同类型 Worker 有不同工具权限	agent_definitions.py:510
权限继承	Worker 启动时继承 AgentDefinition.permissions	agent_tool.py:68
子进程隔离	Worker 在独立进程，不共享 Coordinator 内存	subprocess_backend.py:87
任务追踪	tool_metadata["async_agent_tasks"] 记录所有 Worker	query.py:341
自动等待通知	Coordinator 回合结束后轮询等待 Worker 完成	coordinator_drain.py:159
Worker 崩溃重启	发后续指令时自动重启 Worker 进程	manager.py:335
Team 管理	Team 配置持久化到 team.json	team_lifecycle.py:189
禁止 Worker 递归	Worker 的 disallowed_tools 包含 agent	agent_definitions.py:566
max_turns 限制	Worker 执行回合数有上限	agent_definitions.py:105

子 agent 工作不完全时，但返回给主 agent 已经完成，如何让主 agent 解决此问题，看下源码，该项目如何解决的，在主 agent 中设计 prompt 可以解决吗？

子 agent 工作不完全，但返回已完成，怎么办？

项目的解决方式：

方式	说明
Prompt 要求	Coordinator 写 worker prompt 时，要求“跑测试、跑类型检查、提交后报告 hash”
Coordinator 判断	Coordinator 读 <code><result></code> 的内容，看是否真的完成了
独立验证	派另一个 verification worker 去验收

多个子 agent 协作：

协调方式三：并发控制规则

[coordinator_mode.py:367-370](#):

Manage concurrency:

- **Read-only tasks** (research) — run in parallel freely
- **Write-heavy tasks** (implementation) — one at a time per set of files
- **Verification** can sometimes run alongside implementation on different file areas

任务类型	并发规则	为什么
研究 (只读)	可以并行	不改文件，不会冲突
实现 (写文件)	同一组文件只让一个 agent 操作	防止两个 agent 同时改同一个文件
验证	可以和实现并行 (如果操作不同文件)	验证其他 agent 的代码

问题	解决方式
A 依赖 B 的结果	Coordinator 等 B 完成，把 B 的结果放进 A 的 prompt
A 和 B 都要改同一文件	Coordinator 只让一个先做，另一个等第一个完成
A 和 B 都只读文件	可以并行，Coordinator 同时启动
A 的结果要给 B 用	Coordinator 读 A 的 <code><result></code> ，综合后写进 B 的 prompt

核心原则：

- 子 agent 之间不直接通信
- Coordinator 是唯一的信息中转站
- Coordinator 负责等待 → 综合 → 指派

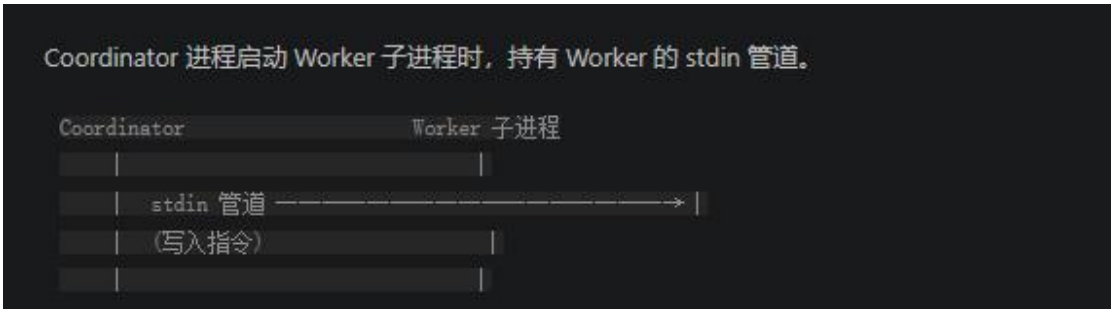
这就像项目经理：

- 开发人员 A 做完了，项目经理看结果
- 项目经理把 A 的结果整理成规格，派给开发人员 B
- B 不需要知道 A 是谁，只需要知道具体要做什么

通信：

1. Stdin/stdout(任务指令通道)：

Coordinator 进程启动 Worker 子进程时，持有 Worker 的 stdin 管道



触发时机：

启动 Worker 时：Coordinator 调用 agent 工具，prompt 通过 stdin 传入

继续 Worker 时：Coordinator 调用 send_message 工具，后续指令通过 stdin 传入

选择原因：

Coordinator 直接持有子进程的 stdin，写入简单；不需要额外的文件系统操作；低延迟，实

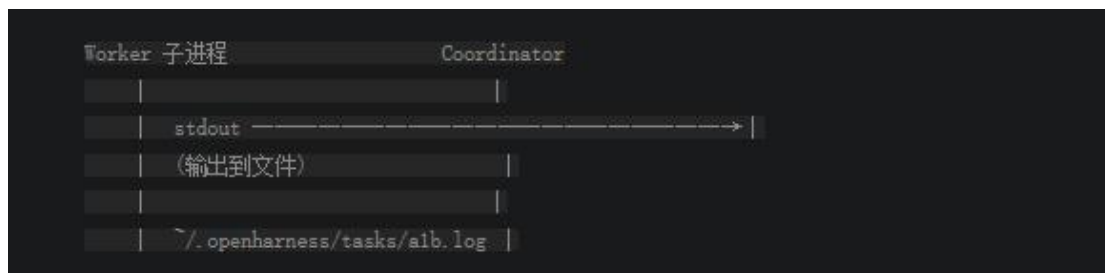
时传递

缺点:

消息不持久化，发完就没了；依赖 Coordinator 进程活着；Worker 崩溃重启后，之前收到的指令丢失

2. 输出文件.log（结果通道）

Worker 子进程的 stdout 被父进程捕获，写到 .log 文件



触发时机:

Worker 执行过程中持续输出，每次输出都被追加到 .log 文件

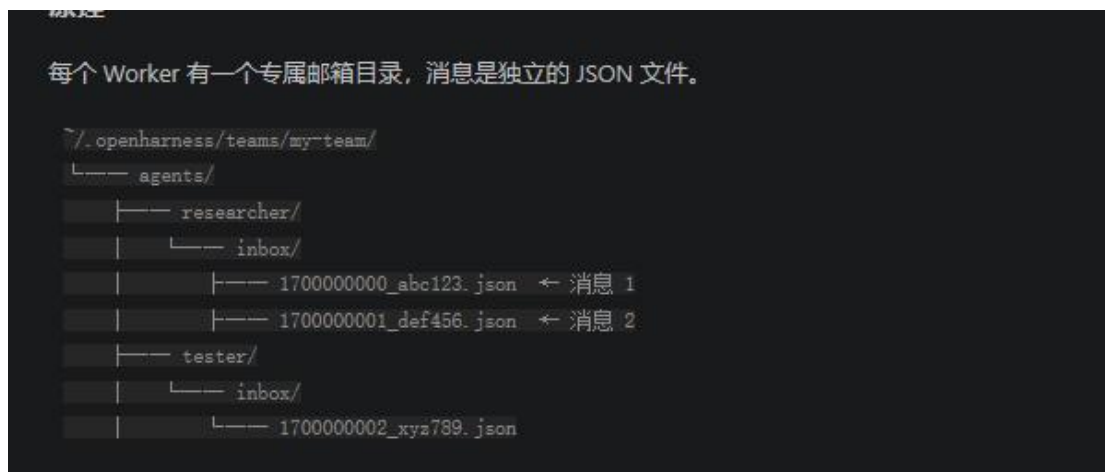
选择原因:

Worker 输出可能很长，不能直接塞进内存；需要持久化，Coordinator 可以随时读取；Worker 完成后，Coordinator 读取文件，包装成 XML 通知

缺点:

不是实时通知，Coordinator 需要轮询；文件可能很大，读取时需要截断

3. MAILBOX（邮箱）



触发时机:

Worker 请求权限: Worker 写 permission_request 到 Leader 邮箱

Leader 批准权限: Leader 写 permission_response 到 Worker 邮箱

Worker 之间通信: Worker A 写消息到 Worker B 邮箱

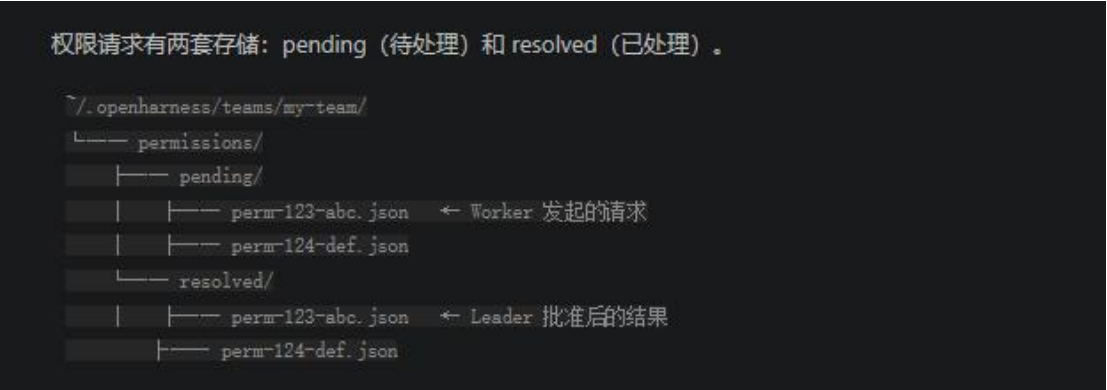
选择原因:

Worker 之间没有直接的管道连接，只能通过文件系统；息持久化，进程崩溃后消息不丢失；支持异步通信，发送者不需要等待接收者

缺点:

不是实时，需要轮询读取；文件系统操作，延迟比 stdin 高

4. Persission 文件，只用于权限



触发时机:

Worker 需要权限: Worker 写请求到 pending 目录

Worker 轮询结果: Worker 查 resolved 目录有没有自己的请求

Leader 处理请求: Leader 读 pending, 处理后写到 resolved

选择原因:

权限请求是阻塞操作, Worker 需要等待结果; 用文件存储, Worker 可以轮询检查结果; pending/resolved 分离, 清晰标记请求状态

和 Mailbox 的关系

Permission 文件和 Mailbox 是两套并行的权限协商方式:

方式	实现	适用场景
Permission 文件	pending/resolved 目录	Worker 自己轮询结果
Mailbox	permission_request/_response 消息	Leader 主动推送结果

为什么权限需要两条通路?

问题场景：Worker重启后怎么办？

场景：Worker请求权限后突然重启了

只有Mailbox通道的问题：

- 1. Mailbox消息在inbox目录
- 2. Worker重启后，inbox消息可能已读/未读状态混乱
- 3. Worker不知道之前发过什么权限请求
- 4. 权限请求丢失

有文件通道的解决方案：

- 1. Worker重启后检查pending目录
- 2. 发现自己之前发过perm-001请求
- 3. 检查resolved目录，看是否已处理
- 4. 如果已处理 → 直接获取结果
- 5. 如果未处理 → 重新通过Mailbox发送请求通知

双重通道的作用：

通道	作用	特点
Mailbox通道	实时通知	快速传递，但不保证持久
文件通道	状态持久化	可靠存储，重启后可恢复

关系：

- Mailbox通道 = “通知系统”（告诉对方有事情）
- 文件通道 = “档案系统”（记录事情的详情和状态）

1.stdin/stdout 详解

一、stdin/stdout管道通道

这是什么

stdin是“标准输入”，stdout是“标准输出”。这是进程间的管道。

Coordinator启动Worker时创建子进程，子进程有stdin管道。Coordinator通过这个管道向Worker写入内容，Worker通过stdin读取内容。

用来传什么

stdin传递两类内容：

第一类：初始prompt

Worker刚启动时，Coordinator通过stdin传入第一条任务。Worker从stdin读取，拿到prompt，开始执行。

第二类：send_message消息

Coordinator用send_message工具继续Worker时，消息通过stdin传入。Worker收到新消息，继续执行新任务。

特点是什么

stdin的特点：

即时性：写入后Worker立即能读，不需要等待。

单向性：主要是Coordinator向Worker发送，Worker很少向stdin写入。

进程内：只存在于Worker进程和Coordinator进程之间，不持久化。

举个例子

Coordinator调用agent创建Worker：

系统启动Worker进程，Worker的stdin管道打开。

系统通过stdin写入：

```
[{"text": "分析src/auth目录，找出Session认证实现细节"}]
```

Worker从stdin读取这一行，解析JSON，拿到text内容。这就是Worker的第一条任务。

Worker开始执行，读文件、分析代码。

完成后，Worker需要返回结果。但Worker不通过stdin返回，Worker用另一个通道。

Worker → Coordinator：通过输出文件 + XML 通知：

工人执行过程中，所有输出写到 .log 文件，完成后通过 mailbox 将内容发送给 coordinator，coordinator 维护一个 pending 和完成的邮件目录。工人完成后，Coordinator 从 .log 文件读取结果，格式化 XML 通知

XML 格式 (在 `coordinator_mode.py:109-126`) :

```
<task-notification>
<task-id>agent-alb</task-id>
<status>completed</status>
<summary>Agent "Investigate auth bug" completed</summary>
<result>Found null pointer in src/auth/validate.ts:42...</result>
<usage>
  <total_tokens>1500</total_tokens>
  <tool_uses>3</tool_uses>
  <duration_ms>2500</duration_ms>
</usage>
</task-notification>
```

意思：工人完成后，Coordinator 读取它的输出，包装成 XML，作为“用户消息”发给 LLM。

2. Mailbox 详解

3. Worker ↔ Worker (同 Team) : 通过 Mailbox

如果多个工人在同一个 Team，它们可以通过 文件邮箱 互相通信。

在 `mailbox.py:102-109`:

```
class TeammateMailbox:
    """File-based mailbox for a single agent within a swarm team.

    每条消息是一个 JSON 文件:
    ~/.openharness/teams/<team>/agents/<agent_id>/inbox/<timestamp>_<id>.json
    """
```

存储结构:

```
~/.openharness/teams/
├── my-team/
│   └── agents/
│       ├── researcher/
│       │   └── inbox/
│       │       ├── 1700000000_abc123.json
│       │       └── 1700000001_def456.json
│       └── tester/
│           └── inbox/
│               └── 1700000002_xyz789.json
```

消息类型 (在 `mailbox.py:27-35`) :

```
MessageType = Literal[
    "user_message",          # 普通消息
    "permission_request",     # 权限请求 (工人问指挥官)
    "permission_response",    # 权限响应 (指挥官回答工人)
    "sandbox_permission_request", # 沙箱权限请求
    "sandbox_permission_response", # 沙箱权限响应
    "shutdown",              # 关闭请求
    "idle_notification",     # 空闲通知
]
```

Harness 设计:

任务追踪: 每次启动工人，都会记录到 `tool_metadata`

自动等待和通知: Coordinator 每次回合结束后，系统自动检查有没有未通知的工人。即查 `tool_metadata` 找所有未通知的工人，问任务管理器"这个工人完成了吗？"完成了就格式化 XML，发给 Coordinator；没完成就每 0.1 秒轮询一次。

工人自动重启: 工人进程死了，`write_to_task` 发现 `stdin` 写不进去，自动重启工人进程，新进程没有之前的上下文，但可以继续执行新的指令，重启前会写入提示，让 Coordinator 知道上下文丢失了

邮箱原子写入: 先获取文件锁，防止并发写冲突，先写 `.tmp` 文件，写完后原子替换成正式文件，读消息的人永远不会看到半写状态。

8. skills 子系统

Skills 加载时的来源：内置技能、用户技能、额外目录的技能、项目技能、插件带来的技能

3. Skill 的目录结构

看 loader.py:153-206:

```
<技能目录>/
├── plan/
│   └── SKILL.md ← 技能定义文件
├── review/
│   └── SKILL.md
├── debug/
│   └── SKILL.md
```

每个技能是一个文件夹，里面有一个 SKILL.md 文件。

这个 MD 文件可以有 YAML frontmatter（前面你学过的）：

```
---
name: plan
description: Design an implementation plan
user-invocable: true
disable-model-invocation: false
model: claude-opus
---

# plan
...手册内容...
```

skills 通过将名称与描述塞进上下文，被 agent 知道。可以用户指定调用 skill，也可以 agent 分析选择调用 skill。agent 调用 skill，就是把手册内容塞进上下文。

总结: Skills 的 7 层 Harness

Harness	代码位置	作用
目录安全检查	loader.py:134	拒绝绝对路径和 <code>..</code> 路径, 防止恶意目录
目录发现顺序	loader.py:83-123	从当前目录往上走, 收集所有技能目录
注册覆盖机制	registry.py:14-18	后注册覆盖前注册, "就近优先"
去重机制	loader.py:167, registry.py:24-29	防止同一技能重复加载/列出
Model 调用限制	context.py:39, skill_tool.py:37	两道检查, 禁止 Agent 自动调用某些技能
Coordinator 隔离	context.py:140-143	子 Agent 不继承技能列表
Settings 控制	loader.py:59	用户可关闭项目技能加载

个人优化:

S（背景）：OpenHarness 是一个开源 Agent Harness 项目，但在上下文压缩系统、工具执行鲁棒性、记忆搜索能力等方面存在多个已知缺陷。例如 `grep/glob` 工具在特定场景下会因管道缓冲阻塞而永久挂起，多工具并行时权限确认弹框互相覆盖导致用户输入被吞掉，记忆搜索在中文场景下效果差且只能匹配元数据。

T（任务）：

1. 定位并修复工具执行中的挂起和死锁问题，提升 Agent 运行稳定性
2. 优化上下文压缩系统，解决大工具结果和视觉场景下的 Token 溢出
3. 改进权限确认流程，保障多工具并行时用户体验
4. 增强记忆系统：支持完整 YAML 解析、正文搜索、中文分词
5. 扩展 Provider 兼容性，接入更多 LLM 后端

A（行动）：

- 工具鲁棒性修复：通过分析 `asyncio` 子进程管道机制，定位到 `grep/glob` 工具因 `rg` 子进程 `stderr` 输出写满 OS 管道缓冲区（64KB）导致阻塞的根因，将 `stderr` 重定向到 `DEVNULL` 而非 `PIPE`；`bash` 工具超时后 `stdout` 流未关闭的修复采用 `asyncio.wait_for(2s)`

保底读取；`session_runner` 死锁通过 `stderr=STDOUT` 合并解决

- 压缩系统优化：① 修复大工具结果（>16K chars）在压缩时的边界处理，超标输出写入 `tool_artifacts` 文件并标记为可微压缩，旧 MCP 结果加入 `context_collapse` 修剪链；② 补充 vision context 的图片 Token 估算（默认 3072 tokens/图），防止多图场景低估触发 API 错误

- 权限串行化：分析并发权限弹框的竞争条件，在 `_ask_permission` 中引入 `asyncio.Lock`，确保同一时刻只有一个确认弹框处于活跃状态，避免多个 `tool_use` 同时触发弹框时的 UI 状态覆盖

- 记忆系统增强：① 替换手动 `---` 分隔符解析为 `yaml.safe_load()`，将 block scalars 等 YAML 标准结构纳入支持；② 搜索从仅匹配元数据扩展为元数据（权重 2x）+ 正文（权重 1x）混合评分；③ 新增 `_tokenize()` 中的 Han 字符识别（一-隼），支持中文查询

- Provider 扩展：实现 OpenAI-compatible API 客户端，包含 `reasoning_content` 字段兼容处理；新增 `qwen/gemini/minimax` 三个内置 Provider Profile

R（结果）：

- 修复 4 个工具挂起/死锁的根因级别 Bug，`grep/glob/bash/session_runner` 稳定性显著提升

- 压缩系统大工具结果边界处理和视觉 Token 估算使多模态长会话不再因 Token 低估而 API 报错

- 权限弹框串行化后并发工具调用场景的确认交互零丢失

- 记忆搜索支持中文后中文查询匹配率显著改善，正文搜索使记忆召回覆盖度提升

- 新增支持 10+ OpenAI-compatible 后端（DashScope/DeepSeek/GitHub Models/Groq/Ollama 等）

优化项 1: grep/glob 工具 stderr 管道阻塞修复

定位并修复 `grep/glob` 工具的 `stderr` 管道缓冲阻塞问题（`grep` 与 `glob` 在运行时需要调用一个 `rg` 子进程，`rg` 子进程因管道满而永久挂起）。

根因分析：`ripgrep`（`rg`）在处理大型仓库时会向 `stderr` 输出大量匹配统计信息。当 `stderr` 配置为 `asyncio.subprocess.PIPE` 时，这些输出写入 OS 管道缓冲区。POSIX 管道缓冲区大小约为 64KB，一旦写满而没有人读取 `stderr`，子进程的 `write()` 系统调用会阻塞，导致进程永久挂起。

修复代码位置：

将 `glob` 工具的本地和 docker 沙箱执行进行问题修复 - 将 `stderr=PIPE` 改为 `stderr=DEVNULL`，丢弃 `stderr` 输出，避免管道阻塞。

将 `grep` 工具的两种搜多模式：普通搜索 + 单文件搜索以及两种执行环境本地+Docker 进行问题修复

同时在 :197 增加了 `limit=8 * 1024 * 1024`（8 MB）的 `asyncio` 流限制，将单行最长限制从 64KB 变为 8MB，防止读取文件的单行超长（如 minified JS 文件）触发 `LimitOverrunError`。

Unix/Linux 的“双通道”设计

每个进程在启动时，默认会打开三个“文件描述符”：

编号	名字	作用	类比
0	stdin	标准输入	就像耳朵，接收别人的话
1	stdout	标准输出	就像嘴巴，输出正常回答
2	stderr	标准错误	就像记事本，输出备注/警告

用大白话说：

stdout	stderr
正式回答	旁白/备注
用户关心的结果	系统统计信息
必须看的内容	可选的内容

ripgrep 的设计选择

ripgrep 选择这样分配输出：

内容	输出到哪里	为什么
搜索结果（file.py:3:hello）	stdout	用户要的结果
统计信息（55 files searched）	stderr	旁白，不是核心结果
错误信息（permission denied）	stderr	警告，不影响结果

这是 ripgrep 的设计选择，不是 Python 控制的。

```
Coordinator
|
| 派 Worker 去研究代码
|
|  Worker 子进程启动
|   |
|   | Worker 收到任务：“找出所有 hello 函数”
|   |
|   | Worker 调用 grep 工具
|   |
|   | grep 工具启动 ripgrep 子进程
|   |   |
|   |   | ripgrep 搜索
|   |   |
|   |   | stdout -> Worker 读取结果 ✓
|   |   |
|   |   | stderr -> Worker 不读取 X ← 问题在这里！
|   |   |
|   |   | stderr 管道满了
|   |   |
|   |   | ripgrep 卡死
|   |   |
|   |   | Worker 卡死
|   |   |
|   |   | Coordinator 等不到 Worker 完成
```

优化项 2：bash 工具超时后 stdout 残留问题：bash 工具超时后 stdout 流未关闭，通过 asyncio.wait_for 保底读取

三、bash 工具的工作流程

```
AI 调用 bash 工具
|
| "帮我运行 npm install"
|
|—— bash 工具启动子进程
|
| 子进程运行命令
|
| npm install 开始执行...
|
| 子进程输出结果到 stdout 管道
|
|—— bash 工具读取 stdout 管道
|
|—— 把结果返回给 AI
```

根因分析：当 bash 命令超时被 kill 后，子进程的 stdout 管道中可能仍有未读数据。如果不读取这些残留数据，下次读取时可能读到"上一轮"的脏数据。另外，如果在终止进程后尝试读取 stdout 内的残留数据，如果没有超时保护，可能无限等待（因为管道可能并没有关闭）

```
命令超时：
|
|—— terminate 子进程
|
|—— 等子进程结束
|
|—— 可能卡在这里（如果子进程不响应 terminate）
|
|—— kill 子进程
|
|—— 读 stdout 残留
|
|—— process.stdout.read()
|
|—— 如果管道关闭了 → 返回空，正常
|
|—— 如果管道没关闭 → **无限等待！**
|
|—— **卡死在这里！**
```

十八、管道不关闭的技术原因

子进程被 kill 时，管道的“写端”可能没有正确关闭。

正常退出流程	被 kill 流程
1. 子进程完成工作	1. 子进程被强制杀死
2. 子进程关闭 stdout	2. 子进程来不及关闭 stdout
3. 管道写端关闭	3. 管道写端可能还开着
4. 管道关闭	4. 管道可能没关闭
5. 父进程 read() 返回空	5. 父进程 read() 可能永远等

十四、改进后的流程

改进后的设计：

命令超时：

```

|
|—— terminate 子进程
|
|—— 等子进程结束（最多 2 秒）← 有超时保护
|
|—— 2 秒内结束 → 正常
|
|—— 2 秒没结束 → TimeoutError → kill 子进程
|
|—— kill 子进程
|
|—— 读 stdout 残留（最多 2 秒）← 有超时保护
|
|—— wait_for(read(), timeout=2.0)
|
|—— 2 秒内读到 → 正常返回数据
|
|—— 2 秒没读到 → TimeoutError → remaining = b""
|
|—— **放弃读取，不会无限等！**
```

优化项 3：session_runner 后台任务死锁修复：后台任务 session_runner 因未读 stderr 管道导致的死锁

根因分析：session_runner 是用来启动后台任务的。session_runner.py 使用 create_shell_subprocess 启动子进程时指定了 stderr=PIPE，即开通了 stderr 管道，但没有独立的 asyncio task 来读取 stderr 管道。当子进程的 stderr 输出超过 OS 管道缓冲区 64KB

时，子进程被阻塞，但父进程在等待 stdout 关闭——形成死锁。

修复代码位置：

- src/openharness/bridge/session_runner.py:44

stderr=asyncio.subprocess.STDOUT, # 将 stderr 合并到 stdout，单一管道避免死锁

优化项 4：权限确认弹框串行化（asyncio.Lock）：引入 asyncio.Lock 串行化机制解决多工具并行时权限确认弹框互相覆盖

根因分析：当 LLM 一次返回多个 tool_use（如同时 Write + Bash），两个工具都需要权限确认时，_ask_permission 会同时被调用两次。如果不加锁，两个弹框的 _emit 会交叉发送到前端，导致 UI 状态混乱——第二个弹框覆盖第一个，用户的 y/n 响应被路由到错误的请求，被覆盖弹框的请求因超时被拒绝。

修复：用 asyncio.lock 锁串行弹框。该锁是一个互斥锁，锁空闲时获取锁，不空闲时等待锁空闲后才可拿去，完成后释放锁

修复代码位置：

- src/openharness/ui/backend_host.py:78 — 锁初始化

self._permission_lock = asyncio.Lock()

- src/openharness/ui/backend_host.py:729-751 — 锁定串行化

async def _ask_permission(self, tool_name: str, reason: str) -> bool:

 async with self._permission_lock: # ← 关键：确保同时只有一个弹框活跃

 request_id = uuid4().hex

 future: asyncio.Future[bool] = asyncio.get_running_loop().create_future()

 self._permission_requests[request_id] = future

 await self._emit(BackendEvent(type="modal_request", modal={...}))

 try:

 return await asyncio.wait_for(future, timeout=300)

 except asyncio.TimeoutError:

 return False

 self._permission_requests.pop(request_id, None)

优化项 5：记忆系统 YAML 解析增强

将 YAML frontmatter 解析从手动逐行拆分升级为 yaml.safe_load。原来依靠手动逐行拆分解析，只能处理简单 key: value，遇到复杂语法就读错。修复后使用专业的 yaml.safe_load 解析器，能正确处理所有 YAML 语法，不会读错。

本项涵盖两处优化：

5a. 记忆文件扫描 YAML frontmatter 解析

- src/openharness/memory/scan.py:28-53 — _parse_memory_file()

解析 --- name: xxx / description: xxx / type: xxx --- YAML frontmatter# 支持 name、description、type 三个标准字段# body_preview 提取正文前 300 字符（排除 frontmatter 和描述行）

5b. Skill 文件加载 YAML 解析（yaml.safe_load）

- src/openharness/skills/loader.py:113

metadata = yaml.safe_load(content[4:end_index])

- 使用 yaml.safe_load 替代手动逐行 key: value 拆分，正确支持 YAML 块标量（>、|）、

引用值等标准结构。

5c. Plugin 文件加载 YAML 解析

- src/openharness/plugins/loader.py:175

```
parsed = yaml.safe_load(raw_frontmatter) or {}
```

5d. Agent 定义 YAML 解析

- src/openharness/coordinator/agent_definitions.py:657

```
parsed = yaml.safe_load(fm_text)
```



```
修复后:

YAML 文件:
---
description: >
  这是一段
  多行描述
---

解析过程:
|
| 找到 frontmatter 区域
|
| 用 yaml.safe_load 解析
|
| yaml.safe_load 认识 ">" 是折叠块里
|
| 正确解析多行内容 ✓

结果: description = "这是一段 多行描述" ← 正确
```

优化项 6: 记忆搜索正文匹配 + 中文分词支持

原来的代码只搜索元数据 (title 和 description) 不搜索正文内容, 导致元数据中没有但正文中其实包含的记忆无法匹配

原来的分词只针对英文, 导致搜索中文内容的记忆时, 无法分词, 即无法匹配中文, 中文内容无法搜索。

为记忆搜索增加正文匹配能力 (此前只匹配元数据), 引入 Han 字符分词支持中文查询。

六、问题1：正文内容找不到

假设记忆文件是这样的：

```
---
name: 架构说明
description: 项目架构记录
---
```

正文：项目使用 PostgreSQL 作为数据库，存储用户数据。

用户搜索“PostgreSQL”：

```
原来只搜索元数据：
|
| 搜索 title: “架构说明” → 没有 “PostgreSQL”
|
| 搜索 description: “项目架构记录” → 没有 “PostgreSQL”
|
| 正文: “项目使用 PostgreSQL ..” → **不搜索！忽略！**
|
| 结果：找不到！ score = 0
|
| **明明正文里有 PostgreSQL，却找不到！**
```

修复：

修复后搜索元数据 + 正文内容，元数据权重更高。

原来	修复后
只搜索 title + description	搜索 title + description + body_preview
正文被忽略	正文也搜索
元数据和正文同等权重	元数据权重 2x，正文权重 1x

元数据为简介，若包含大概率就是；正文内容为详细内容，即使有也可能只是提到一下，所以元数据权重更高

```
# 修复后的代码（搜索元数据 + 正文）
def find_relevant_memories(query, cwd):
    tokens = _tokenize(query) # 分词（支持中文）

    for header in scan_memory_files(cwd):
        meta = f"{header.title} {header.description}".lower() # 元数据
        body = header.body_preview.lower() # 正文

        # 元数据匹配权重 2x，正文权重 1x
        meta_hits = sum(1 for t in tokens if t in meta)
        body_hits = sum(1 for t in tokens if t in body)

        score = meta_hits * 2.0 + body_hits # ← 元数据权重 2 倍
```

增加中文分词，以每个中文 token 为维度分割。

优化项 7：压缩系统大工具结果边界处理

修复大工具结果（>16K chars）在压缩时的边界处理，超标输出（默认为 16000）写入 tool_artifacts 文件，将工具结果预览（3000 字符）+ 文件路径发送给 tool_metadata + messages。涉及代码位置：

- 大结果卸载：src/openharness/engine/query.py:472-501 — _offload_tool_output_if_needed()
inline_limit = tool_output_inline_chars() # 默认 16,000 chars
if len(output) > inline_limit:
 return output, None # 小结果直接内联 # 大结果：写入
tool_artifacts/{timestamp}-{tool}-{uuid}.txt
artifact_path = _tool_artifact_dir() / f'{timestamp}-{name}-{uuid}.txt'
- 工具产物目录：src/openharness/engine/query.py:461-464
def _tool_artifact_dir() -> Path:
 artifact_dir = get_data_dir() / "tool_artifacts"
 artifact_dir.mkdir(parents=True, exist_ok=True)
- 微压缩标记：src/openharness/services/tool_outputs.py — is_microcompactable_tool_result()
判断哪些工具结果可以被微压缩清除
- Context Collapse 文本截断：src/openharness/services/compact/init.py:288-294 — _collapse_text()
将超长文本截断为 head(900) + [collapsed N chars] + tail(500)

优化项 8：视觉上下文 Token 估算

补充 vision context 的图片 Token 估算（默认 3072 tokens/图），防止多图场景下，未预留图片 token 从而导致上下文超长，触发 API 错误。

压缩时，将图片内容替换为占位符